ROBOTICS SOFTWARE STATE OF PRACTICE

A STATE-OF-THE-PRACTICE STUDY OF ROBOTICS SOFTWARE FOR MOTION PLANNING AND SIMULATION

BY

ZIYANG FANG, B.Eng.

A Report

submitted to the Department of Computing and Software

and the School of Graduate Studies

of McMaster University

in partial fulfilment of the requirements

for the degree of

Master of Engineering

Master of Engineering (YYYY) McMaster University

(Department of Computing and Software) Hamilton, Ontario, Canada

TITLE:	A State-of-the-Practice Study of Robotics Software for Motion Planning and Simulation
AUTHOR:	Ziyang Fang B.Eng. (Mechanical Design, Manufacturing and Automation), Fuzhou University, School of Mechanical Engineering, China
SUPERVISOR:	Spencer Smith Matthew Giamou
NUMBER OF PAGES:	xvii, 112

Lay Abstract

Abstract

Your Dedication

Optional second line

Acknowledgements

Contents

List of Figures

List of Tables

Notation, Definitions, and Abbreviations

Notation

CR	Consistency ratio (Analytic Hierarchy Process)
CI	Consistency index (Analytic Hierarchy Process)
$\lambda_{\max}$	Principal eigenvalue of a pairwise-comparison matrix

Definitions

Alive package

In this report, a package is considered alive if it has received any commit or release in the 18 months preceding the measurement date, following the Domain X rule.

Software family

A set of programs whose common properties are extensive enough

that studying them collectively is advantageous before analysing individual members Parnas [1976].

State-of-the-Practice study

An assessment of the artifacts, tools, processes, and quality of existing software in a chosen domain, in contrast to a study that designs or implements new software.

Surface measure

An assessment based on shallow but systematic checks performed within a limited time budget rather than on exhaustive experimental evaluation.

Abbreviations

AHP	Analytic Hierarchy Process
API	Application Programming Interface
CI	Continuous Integration
CI/CD	Continuous Integration and Continuous Delivery
DART	Dynamic Animation and Robotics Toolkit
DDS	Data Distribution Service
GIS	Geographic Information Systems
LBM	Lattice Boltzmann Method

LLM	Large Language Model
MI	Medical Imaging
MJCF	MuJoCo XML configuration format
MPS	Motion Planning and Simulation
OMPL	Open Motion Planning Library
OSS	Open Source Software
RL	Reinforcement Learning
ROS	Robot Operating System
SCC	Source-code line counter used for measurement
SDF	Simulation Description Format
SDK	Software Development Kit
SOFA	Simulation Open Framework Architecture
SOP	State-of-the-Practice
URDF	Unified Robot Description Format
YARP	Yet Another Robot Platform

Declaration of Academic Achievement

The student will declare his/her research contribution and, as appropriate,

Chapter 1

Introduction

Robotics software for motion planning and simulation (MPS) enables researchers and engineers to model robot kinematics and dynamics, simulate sensor data, plan collision-free trajectories, and test control strategies before deploying to physical hardware. In practice, this software is central to experimental reproducibility and to the transition from simulation to physical deployment, so its engineering quality has direct consequences for both research and operational outcomes.

These consequences make software quality a central concern for motion planning and simulation software. A simulator that produces incorrect trajectories, or a planner that crashes under realistic inputs, can compromise experimental results in ways that are not immediately obvious. Qualities such as correctness, reliability, and long-term sustainability are not guaranteed by domain expertise alone.

Software engineers have developed methods and tools for improving software quality: version control, automated testing, and continuous integration are standard practice in professional contexts. There has historically been a gap, however, between these practices and what researchers actually do when developing scientific software.

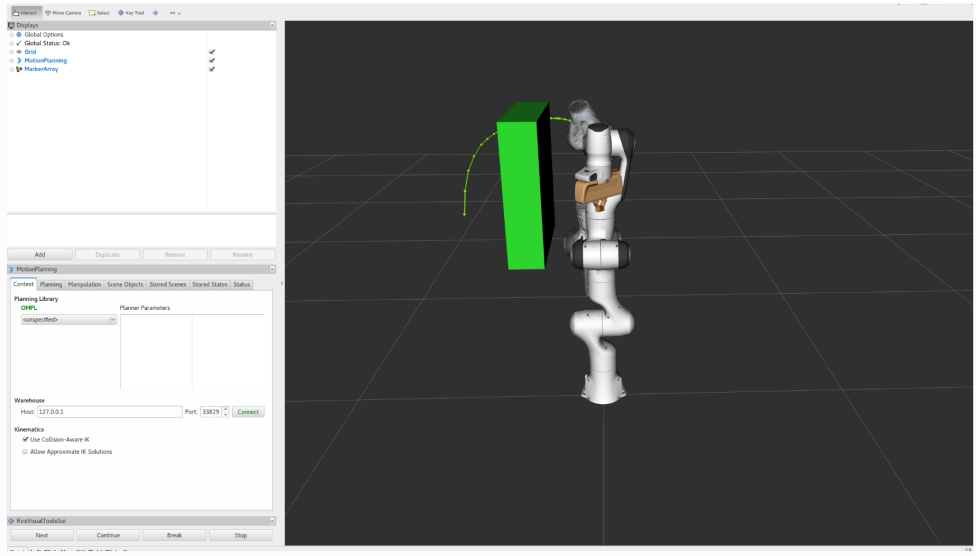

Figure 1.1: An illustrative path planning task: a manipulator computing a collision-free path around an obstacle.

Scientists and engineers working on research software tend to learn software engineering skills informally, picking them up from colleagues or through trial and error rather than through formal training Hannay et al. [2009]. Hannay et al. [2009] observe that many scientists show indifference to standard software engineering concepts, and Prabhu et al. [2011] report that more than half of the scientists they surveyed did not use a debugger. Because much of the software studied in this report originates in research settings, these concerns apply directly to the domain.

Previous State-of-the-Practice studies have investigated this gap across several scientific software domains, including geographic information systems, mesh generation, seismology, statistical software for psychology, medical imaging software, and Lattice Boltzmann method software Smith et al. [2018c, 2016, 2018b,a, 2024a,b]. This report applies the same methodology Smith et al. [2021] to MPS software. The aim is to understand how MPS software holds up against established software engineering

practices and how it compares with other research software domains. The comparison identifies where MPS developers already meet those practices and where gaps remain. The report ranks the selected packages according to the defined quality attributes; those rankings describe the current state of the practice, not a recommendation for any particular robotics task or context.

This chapter is organized as follows. Section 1.1 states the research questions. Section 1.2 defines the scope of the study. Section 1.3 provides a high-level overview of the methodology. Section 1.4 describes the contributions of the report. Section 1.5 outlines the remaining chapters.

1.1 Research Questions

The following research questions guide this study. MPS, defined above, refers throughout to the studied robotics software domain.

RQ1. What software packages are candidates for the State-of-the-Practice study of MPS software?

RQ2. What is the ranking of the candidate packages from RQ1 with respect to the software qualities of installability, correctness and verifiability, reliability, robustness, usability, maintainability, reusability, understandability, and visibility and transparency?

RQ3. How do MPS projects compare to other research software domains with respect to the artifacts present in their repositories?

RQ4. How do MPS projects compare to other research software domains with respect

to the use of tools?

RQ5. How do MPS projects compare to other research software domains with respect to the principles, processes, and methodologies used?

RQ6. What are the pain points for developers working on MPS software projects?

RQ7. How do the pain points for MPS developers compare to the pain points for developers in other research software domains?

RQ8. For MPS developers, what specific best practices address the pain points identified in RQ6 and the software quality concerns identified in RQ2?

RQ9. What research software development practices, beyond those already used by MPS developers, address the pain points identified in RQ6?

RQ1 covers candidate discovery; RQ2 measures the selected packages from public artifacts. RQ3–RQ5 compare MPS software with prior State-of-the-Practice studies of other research software domains. RQ6–RQ9 concern the developer perspective on pain points and best practices, and are conditional on approved interview data being available.

1.2 Scope

This study covers MPS software: robotics packages for motion planning, dynamics, physics, simulation, and supporting middleware. To be included, a package must expose sufficient public evidence, such as a source repository, documentation, issue tracker, releases, or project metadata, to support consistent repository-based measurement. Tools whose core components are proprietary or otherwise closed therefore

fall outside the measured set even when they are widely used in robotics practice. Chapter 3 states the full inclusion and exclusion criteria and gives the rationale for individual tools that were considered and removed.

The selected set is deliberately heterogeneous. Full simulators sit alongside middleware (e.g., ROS 2), planning libraries (e.g., OMPL), and integration frameworks (e.g., MoveIt!), because MPS workflows routinely combine these roles. The comparison therefore treats the selected packages as related members of a domain, not as interchangeable products.

1.3 Overview of the Methodology

This report adapts the State-of-the-Practice methodology Smith et al. [2021], which has previously been applied to domains including geographic information systems, mesh generation, seismology, statistical software for psychology, medical imaging, and Lattice Boltzmann method software Smith et al. [2018c, 2016, 2018b,a, 2024a,b]. The scope and exclusions were reviewed with a domain expert to reduce the risk of misrepresenting the domain. At a high level, the methodology consists of the following steps:

1. Define the MPS software domain and selection criteria.
2. Identify candidate packages from academic sources, community-curated lists, and auxiliary LLM-assisted discovery.
3. Filter the candidate list using domain relevance, public measurability, and repository-based evidence.

4. Measure the selected packages using repository artifacts, documentation, releases, issue trackers, and project metadata.

5. Analyze the selected packages as a software family through commonality analysis.

6. Compare the MPS findings with prior State-of-the-Practice studies of other research software domains.

7. If approved interview data are available, supplement the artifact-based results with developer perspectives on pain points and best practices.

Chapter 3 describes the full candidate discovery, filtering, and measurement procedures.

1.4 Contributions

This report makes the following contributions:

1. A scoped State-of-the-Practice study of MPS software, applying the peer-reviewed methodology of Smith et al. [2021] to a new domain.

2. A transparent candidate discovery and selection process that combines academic sources, community-curated lists, and auxiliary LLM-assisted discovery, with explicit discussion of the role and limitations of each source type.

3. A public-artifact measurement dataset for 28 selected software packages, covering repository evidence, documentation, releases, issue trackers, testing and continuous-integration evidence, and project metadata, organized across the

nine quality categories defined in the Domain X measurement template Smith et al. [2021].

4. A commonality analysis identifying shared capabilities, variability points, and parameters of variation across the selected packages.

5. A comparison of MPS software with findings from prior State-of-the-Practice studies in other research software domains Smith et al. [2024b,a, 2018b], where supported by available evidence.

These contributions are descriptive and methodological. The report characterizes existing software by measuring it against defined quality attributes. The resulting rankings describe the current state of the practice; they do not prescribe one tool as best for every robotics task.

1.5 Organization

The remainder of this report is organized as follows. The research questions are not confined to a single chapter; they are answered throughout the body, and each chapter notes the research questions it addresses to maintain traceability back to Section 1.1.

- **Chapter 2** introduces the robotics software domain and the motion planning and simulation context, reviews prior State-of-the-Practice studies, defines the software quality attributes, provides background on commonality analysis and the Analytic Hierarchy Process, and discusses the limitations of open-source repository-based measurement.

- **Chapter 3** describes the methodology: domain selection, candidate discovery from multiple source types, filtering criteria, the measurement procedure using the Domain X template Smith et al. [2021], the commonality analysis procedure, and the developer interview component with its adapted Domain X interview protocol, recruitment, and analysis plan.

- **Chapter 4** presents the 28 selected packages as a software family and reports the commonality analysis: shared capabilities, variability points, and parameters of variation.

- **Chapter 5** ranks the selected packages on the nine Domain X quality categories, reports the measurement results and repository metrics, and compares the quality-based ranking with community-facing indicators of visibility and adoption.

- **Chapter 6** compares MPS software with prior State-of-the-Practice studies of other scientific software domains on three dimensions: repository artifacts, tool usage, and development principles, processes, and methodologies. The third dimension will draw on developer interview data once approved interviews become available.

- **Chapter 7** reports the developer pain points for MPS software: a summary of the pain points, what developers do about them, and recommended practices. Because no approved interview data is currently available, interview findings are reserved for a future study.

- **Chapter 8** discusses threats to validity, including limitations related to candidate selection, public-artifact measurement, source bias, and the interpretation

of quality scores.

- **Chapter 9** summarizes the study, answers the research questions, states the key findings, and identifies directions for future work.

Chapter 2

Background

This chapter sets out the concepts the rest of the report relies on. Section 2.1 introduces the domain of MPS. Section 2.2 defines the software categories relevant to this study. Section 2.3 reviews the State-of-the-Practice methodology and its prior applications. Section 2.4 defines the nine software quality attributes measured in this study. Section 2.5 introduces commonality analysis as a method for describing software families. Section 2.6 introduces the Analytic Hierarchy Process used to support ranking. Section 2.7 discusses the role and limitations of open-source repository-based measurement.

2.1 Robotics Software for Motion Planning and Simulation (MPS)

MPS software supports the design, testing, evaluation, and deployment of robotic systems. Motion planning software generates feasible robot motions under constraints

such as geometry, kinematics, dynamics, collisions, and task requirements LaValle [2006]. Simulation software provides virtual environments for representing robots, sensors, actuators, controllers, and physical interactions before, or alongside, work with physical hardware Collins et al. [2021].

The domain in this study is broader than a single class of software. It spans full simulation environments (Gazebo, Webots, CoppeliaSim, CARLA, and similar tools), physics and dynamics engines (Bullet, MuJoCo, ODE, DART), motion planning libraries such as OMPL, manipulation frameworks such as MoveIt!, middleware such as ROS 2 and YARP, and a range of supporting development tools. These systems often appear together in a single robotics workflow even when they are not direct substitutes for one another: a simulator may depend on a physics engine for contact and dynamics computation, a planning stack may rely on a collision-checking library and communicate through middleware, and middleware may then connect planning, control, perception, and simulation components into one integrated system.

This breadth creates a scope challenge for a State-of-the-Practice study. A narrow definition restricted to full simulators would make comparison simpler, but it would exclude software that practitioners routinely use in motion planning and simulation workflows. A broader definition better reflects the practical ecosystem, provided the report is explicit about the different roles that software packages play within the domain.

2.2 Software Categories

Assessing software packages requires understanding the licensing model under which each package is distributed. In particular, the availability of source code determines

whether repository-based measurement is possible. Following the terminology used in prior State-of-the-Practice studies Smith et al. [2024a,b], this report distinguishes three common software categories:

Open Source Software (OSS): For OSS, the source code is openly accessible. Users have the right to study, change, and distribute it under a license granted by the copyright holder. For many OSS projects, the development process relies on collaboration among contributors worldwide. Accessible source code exposes the underlying logic of software functions, the development practices used to produce them, and the flaws and potential risks in the final product. OSS is therefore suitable for the kind of repository-based analysis performed in this study.

Freeware: Freeware is software that can be used free of charge. Unlike OSS, the authors of freeware do not permit access to or modification of the source code. To many end users, the distinction between freeware and OSS may not matter. However, for researchers seeking insight into software development processes, inaccessible source code is a barrier to measurement.

Commercial Software: Commercial software is software developed by a business as part of its business. Typically, commercial software requires payment to access all features and does not include access to the source code. Some commercial software is available free of charge, but the lack of source access prevents the kind of repository-based assessment used in this study.

This study assesses only software that fits under the Open Source Software category. This restriction is a methodological necessity, not a judgement about the quality

or importance of freeware or commercial tools.

2.3 State-of-the-Practice Studies

A State-of-the-Practice study examines existing tools, artifacts, processes, and evidence in a chosen domain. Unlike a typical implementation project, the goal is not to design a new system but to learn from existing software by systematically collecting and organizing public evidence. Claims about development activity, documentation quality, issue management, installation behaviour, or maintainability must be supported by observable project artifacts. When evidence is missing or ambiguous, the limitation is recorded rather than filled with assumptions.

The methodology used in this report is based on "Methodology for Assessing the State of the Practice for Domain X" by Smith et al. [2021]. It is designed to be generic, so that it can be applied to any research software domain, and to remain feasible for a small team working within roughly 160 person-hours per domain. The prescribed process runs from domain selection through measurement and analysis: identify a suitable domain and engage a domain expert; construct and filter a candidate software list; perform a domain analysis; gather source code, documentation, and repository metrics for the selected packages; apply a measurement template covering nine quality categories; survey developers through semi-structured interviews where ethics approval permits; use the Analytic Hierarchy Process (AHP) to support cross-quality ranking; and answer a set of research questions about the domain.

This methodology builds on prior State-of-the-Practice assessments conducted across several research software domains: Geographic Information Systems Smith et al. [2018c], Mesh Generators Smith et al. [2016], Seismology software Smith et al.

[2018b], and Statistical software for psychology Smith et al. [2018a]. More recently, the methodology was refined and applied to Medical Imaging (MI) software Smith et al. [2024a] and Lattice Boltzmann Method (LBM) software Smith et al. [2024b], Michalski [2021]. These studies are the primary structural and stylistic references for this report.

The MI and LBM studies follow a common organizational pattern: introduction and motivation, background on the domain and methodology, detailed methodology description, measurement results organized by quality category, comparison with community rankings, developer interviews and pain points, discussion of research questions, recommendations, and conclusions. This report adopts the same general structure while adapting the content to the MPS domain.

Key features of the Domain X methodology that distinguish it from ad hoc software comparisons include: (i) the involvement of a domain expert to vet the software list, domain analysis, and ranking; (ii) a transparent candidate discovery process using multiple source types; (iii) a standardized measurement template that applies the same set of fields (approximately 108, counting substantive questions and supplementary entries together) to every package; (iv) the use of repository-based metrics alongside qualitative assessments; and (v) a commonality analysis that treats the selected packages as members of a software family rather than as unrelated products.

2.4 Software Quality Attributes

Quality is defined as a measure of the excellence or worth of an entity. Following common practice in software engineering, this study treats quality as a collection of distinct attributes, often called “ilities” Smith et al. [2021]. This study assesses nine

quality attributes drawn from the Domain X Appendix B measurement template. Several of the qualities use the word "surface" to indicate that the assessment is based on shallow but systematic checks performed within a limited time budget, not on exhaustive experimental evaluation Smith et al. [2024a].

Installability: The effort required for the installation and uninstallation of software in a specified environment. Measured through the presence and quality of installation instructions, the number of steps required, the availability of automation, the handling of dependencies, and the presence of validation procedures.

Correctness and Verifiability: A program is correct if it matches its specification, which may be stated explicitly or implicitly. The related quality of verifiability is the ease with which the software components or the integrated product can be checked to demonstrate correctness. Measured through the presence of requirements or theory documentation, techniques used to build confidence in correctness (such as automated testing, assertions, or continuous integration), and the availability and quality of tutorials with expected outputs.

Surface Reliability: The probability of failure-free operation of a computer program in a specified environment for a specified time. Surface reliability is assessed through whether the software breaks during installation or initial tutorial use, whether descriptive error messages are displayed, and whether recovery is possible.

Surface Robustness: Software possesses robustness if it behaves reasonably when it encounters circumstances not anticipated in the requirements specification,

and when users violate the assumptions in its requirements specification. Surface robustness is assessed through whether the software handles unexpected or malformed input gracefully, including edge cases such as modified line endings in text input files.

Surface Usability: The extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use. Surface usability is assessed through the presence of getting-started tutorials, user manuals, documented user characteristics, and the availability and variety of user support channels.

Maintainability: The effort with which a software system or component can be modified to correct faults, improve performance or other attributes, or satisfy new requirements. Assessed through version information, contribution and review guidance, available non-code artifacts, issue tracking practices, the percentage of identified issues that are closed, the percentage of code that consists of comments, and the version control system in use.

Reusability: The extent to which a software component can be used with or without adaptation in a problem solution other than the one for which it was originally developed. Assessed through the number of code files and the presence and quality of API documentation.

Surface Understandability: The capability of the software product to enable the user to understand whether the software is suitable, and how it can be used for particular tasks and conditions of use. Surface understandability is assessed

through a review of randomly selected source files, examining indentation consistency, coding standards, identifier quality, use of named constants, comment clarity, algorithm references, parameter ordering, and modularization.

Visibility and Transparency: The extent to which all steps of a software development process and the current status of that process are conveyed clearly to external observers. Assessed through whether the development process is defined and documented, whether the development environment is documented, and whether release notes are published.

These nine qualities organize public evidence into comparable categories. They are not complete evaluations of each package. Scores may vary with the use case, platform, user background, or measurement date. Later chapters therefore report both the measurements and their limitations.

2.5 Commonality Analysis

A domain analysis consists of systematically identifying and documenting the commonalities, variabilities, and parameters of variation of a software family Weiss [1997]. A software family is defined as "a set of programs whose common properties are so extensive that it is advantageous to study the common properties of the programs before analyzing individual members" Parnas [1976]. Weiss [1997] defines commonality analysis as an approach to defining a family by identifying three kinds of information:

Commonalities: Goals, theories, models, definitions, and assumptions that are shared across family members. For example, all robotics simulators share the goal of representing robot kinematics and dynamics in a virtual environment.

Variabilities: Goals, theories, models, definitions, and assumptions that differ between family members. For example, simulators may differ in the physics engines they use, the sensor models they support, or the robot platforms they can represent.

Parameters of Variation: The possible values for each variability, along with their binding time. Binding time records when a particular variation is fixed: at specification time (when the software is designed), at build time (when the software is compiled), at configuration time (when the software is set up for use), or at run time (when the software is executing).

Commonality analysis was added to the Domain X methodology after early applications revealed that software packages within a domain can vary considerably, even when they appear to belong to the same family Smith et al. [2024b]. Without it, comparisons risk treating fundamentally different types of software as if they were interchangeable. The analysis records where the packages do similar things and where their roles diverge, so a low score on, for example, installability means different things for a full simulator and a header-only dynamics library.

In this report, commonality analysis is particularly important because the selected set is deliberately heterogeneous: it includes full simulators, physics engines, planning libraries, and middleware. The commonality analysis identifies shared concerns across these categories (such as robot model representation, integration with external tools, and documentation practices) while also documenting the dimensions on which packages differ.

2.6 Analytic Hierarchy Process

The Domain X methodology uses the Analytic Hierarchy Process (AHP) to combine the nine quality impression scores into an overall ranking that reflects the relative importance of each quality Saaty [1980]. AHP is a multi-criteria decision-making technique developed by Saaty in the 1970s. It structures a decision problem as a hierarchy with a goal at the top (here, overall software fitness), criteria in the middle (the nine measured qualities), and alternatives at the bottom (the 28 measured packages). The relative importance of the criteria is elicited through pairwise comparisons rather than asking the decision-maker to assign absolute weights, which is generally easier and more reliable in practice.

For each pair of criteria, the decision-maker expresses a judgement on Saaty's fundamental scale. The odd values carry the named intensities: 1 means equally important, 3 moderately more important, 5 strongly more important, 7 very strongly more important, and 9 extremely more important. The even values (2, 4, 6, 8) cover intermediate cases between the named intensities. The judgements populate a square reciprocal matrix whose principal right eigenvector, normalized to sum to one, yields the criterion weights. Combining these criterion weights with the per-criterion alternative scores gives an overall ranking of the alternatives.

AHP includes a built-in consistency check. The consistency index $CI = (\lambda_{\max} - n)/(n-1)$, where $\lambda_{\max}$ is the principal eigenvalue of the comparison matrix and n is the matrix dimension, is divided by a random-index value RI tabulated for matrices of each size to produce the consistency ratio $CR = CI/RI$. Saaty's convention is that $CR \leq 0.10$ indicates acceptable consistency; larger values suggest that the pairwise judgements should be revisited. This check is one reason AHP is preferred

in Domain X over a simple unweighted mean of the per-quality scores: the mean treats all qualities as equally important regardless of context, and offers no mechanism for the assessor to make that weighting explicit or to test whether the weighting is internally coherent.

In a State-of-the-Practice study, AHP is an analysis aid for cross-quality comparison under a stated weighting, not a universal claim that one package is best for every use case. The same per-package measurements can yield different rankings under different reasonable weightings, which is why prior studies in the Domain X line report a sensitivity analysis alongside the headline ranking. Chapter 3 describes how AHP would be applied to the measurements in this study; the present report does not compute AHP rankings, because they require a defensible criterion-weighting elicitation that has not been completed at the time of writing. Chapter 5 therefore presents the per-package quality-mean ranking and an explicit comparison with community indicators in place of an AHP-weighted ranking.

2.7 Open Source and Repository-Based Measurement

The measurement process in this study depends on evidence from public artifacts. Public repositories can reveal commit history, release activity, issue tracking, documentation, source code organization, testing practices, dependencies, and community interaction. Public documentation and project websites can add evidence about installation procedures, usage patterns, supported platforms, and design goals. This

dependence on public evidence makes open-source availability a prerequisite for inclusion in the selected set.

Repository-based measurement has inherent limitations. Repository metadata represents a snapshot in time: last commit dates, issue counts, release information, and pull request activity can change after measurement. Some projects distribute their development activity across multiple repositories or use external issue trackers, making it difficult to capture a complete picture from any single repository. The interpretation of repository metrics requires care: a high number of open issues may indicate an active user community rather than poor maintenance, and a low commit frequency may reflect project maturity rather than abandonment.

For these reasons, the measurement record must include clear documentation of measurement dates, repository choices, and interpretation rules. The dependence on public evidence also affects the composition of the selected set. Proprietary tools, or tools with closed core components, may be important in robotics practice but cannot be measured with the same procedure as open-source tools. Chapter 3 discusses the specific inclusion and exclusion decisions that result from this constraint.

Chapter 3

Methodology

This chapter describes the methodology used to conduct the State-of-the-Practice study of MPS software and addresses RQ1 (candidate identification) by documenting how the candidate set was assembled. The methodology follows Smith et al. [2021] and adapts it to the current domain. Section 3.1 describes the domain selection process. Section 3.2 describes the role of the domain expert. Section 3.3 describes the three-source candidate discovery process. Section 3.4 describes the mention-count ranking method. Section 3.5 describes the filtering criteria applied to the candidate list. Section 3.6 presents the selected set of 28 packages. Section 3.7 describes the measurement procedure using the Domain X template. Section 3.8 describes the commonality analysis procedure. Section 3.9 describes the planned developer interview component.

3.1 Domain Selection

The first step of the Domain X methodology is to identify a suitable research software domain. To be applicable for the methodology, the chosen domain must meet four criteria Smith et al. [2021]: (i) it must have a well-defined and stable theoretical underpinning, ideally supported by standard textbooks; (ii) there must be a community of researchers and practitioners studying the domain; (iii) the software packages in the domain must include open-source options; and (iv) a preliminary search should suggest that there are close to 30 candidate software packages that qualify as research software.

The selected domain is *MPS (motion planning and simulation)*. Motion planning and simulation are established subfields of robotics with well-defined theoretical foundations LaValle [2006], Choset et al. [2005]. There is an active international research community, regular conferences (such as ICRA, IROS, and RSS), and a substantial body of published work. A preliminary search confirmed the existence of numerous open-source software packages spanning simulation environments, physics engines, dynamics libraries, motion planning libraries, and robotics middleware.

The domain boundary is broader than "robotics simulators" alone. Many robotics workflows combine a simulator with planning libraries, physics engines, middleware, and supporting tools. Restricting the study to full simulators would make comparison simpler but would omit software that belongs to core motion planning and simulation workflows.

Earlier project discussions considered a narrower domain focused on inverse kinematics software. After consultation with the supervisor and domain expert, the scope

was broadened to the current domain, which provides a richer set of candidate packages and better reflects the interconnected nature of robotics software ecosystems.

3.2 Interaction with the Domain Expert

The Domain X methodology emphasizes that a domain expert is an essential member of the assessment team. Pitfalls exist if non-experts attempt to produce an authoritative list of software or definitively rank packages without domain knowledge. Non-experts must rely on information available online, which has two known drawbacks: (i) online resources may contain false or inaccurate information; and (ii) online resources may omit relevant information that is so ingrained with experts that nobody thinks to record it explicitly Smith et al. [2021].

Domain experts may be recruited from academia or industry. The only requirements are knowledge of the domain and a willingness to be engaged in the assessment process. The domain expert does not need to be a software developer, but they should be a user of domain software. Given that domain experts are likely to be busy people, the methodology is designed to minimize the burden on their time Smith et al. [2024a].

For this study, the domain expert is Dr. Matthew Giamou of the Department of Computing and Software at McMaster University. Dr. Giamou's research includes robotics, motion planning, and state estimation. He reviewed the candidate software list, confirmed that the scope was appropriate, and provided guidance on inclusion and exclusion decisions.

The domain expert was asked to review the candidate software list and provide feedback on scope and coverage. Dr. Giamou confirmed that the list was reasonable

and that most of the shortlisted tools were familiar to him. He noted that several packages (ROS 2, MoveIt!, and OMPL) are not full simulators but can fit within a coherent narrative if their roles are explained carefully. He also confirmed that excluding MATLAB/Simulink, Unity, NVIDIA Isaac Sim, and RaiSim on methodological grounds (closed source or non-OSS licensing for core components, as detailed in Section 3.5.1) was acceptable and would not misrepresent the domain. MRDS and USARSim, which are also excluded, were filtered on independent activity grounds (Section 3.5.3) rather than as a separate expert decision.

3.3 Candidate Software Discovery

Candidate software packages were identified from three types of sources: academic survey papers, community-curated software lists, and LLM-generated software lists. Using multiple source types reduces dependence on any single search method and provides broader coverage of the domain. The collected names were consolidated to merge equivalent entries (for example, "V-REP" and "CoppeliaSim" refer to the same package), and the consolidated list was used as input to the mention-count ranking described in Section 3.4.

3.3.1 Academic Sources

Academic sources identify software packages that appear in research discussions of robotics simulators, simulation tools, and related challenges. These sources included

survey papers and comparative studies that discuss packages such as Gazebo, Webots, V-REP/CoppeliaSim, MuJoCo, and PyBullet in the context of robotics research. Academic sources connect software packages to published research contexts and provide a research-facing view of the domain.

Academic sources have known limitations as the sole basis for candidate discovery. Survey papers tend to emphasize packages relevant to a specific research question, robot type, or time period; some active tools may not appear in any of the surveys read, while older tools sometimes remain visible because they were prominent when a survey was written. That bias is why academic sources were combined with community-curated and LLM-assisted sources rather than used in isolation.

3.3.2 Community-Curated Sources

Community-curated sources broaden the view of robotics software practice beyond what appears in academic surveys. The source catalogue included GitHub topic pages, "awesome" lists (curated collections such as `awesome-robotics-libraries` and `awesome-robotics-simulation`), robotics software directories, best-of lists, and other community-maintained aggregations. Sources covered topics including motion planning, robotics simulation, robotics libraries, ROS 2 ecosystem tools, multibody dynamics simulation, and general robotic tooling.

These sources collect software that practitioners and community contributors consider relevant. They can reveal projects that are underrepresented in academic surveys and can show relationships between simulators, planning libraries, middleware, physics engines, and tools. Community-curated sources also vary in maintenance

quality, inclusion criteria, and update frequency. They were therefore used as discovery aids rather than as indicators of software quality.

3.3.3 LLM-Assisted Sources

LLM-generated software lists were used as an auxiliary source for candidate discovery. This modifies the original Domain X methodology, which prescribes search engine queries and domain expert consultation as the primary discovery methods. The modification is recorded explicitly so that the methodology remains transparent and reproducible in principle.

The reason for including LLM-assisted discovery is practical: large language models can surface software names that appear across a wide range of public information, functioning as a broad aggregation mechanism over existing public data. However, LLM output can also be incomplete, outdated, or incorrect. Names obtained from LLM-generated lists therefore require consolidation, filtering, and verification against public evidence before they can support any claims in this report. In this project, LLM-assisted sources contributed to candidate discovery but did not determine the selected set, and they were never treated as authoritative.

This approach was discussed with the supervisor, who noted that while the use of LLMs is technically a modification of the peer-reviewed methodology, it is defensible for two reasons: (i) LLMs were used together with academic and community-curated sources, not as the sole discovery method; and (ii) LLMs can be understood as a large-scale aggregation mechanism over existing public information, analogous in effect to a broad search over indexed web content.

3.4 Mention Count Ranking

After consolidating equivalent names across the collected sources, each unique software package was counted once per source to produce a mention count. A package that appeared in five academic papers and three community-curated lists would receive a mention count of eight. The consolidated ranking was recorded during the discovery phase and is a transparent input to the filtering process.

The top entries in the consolidated ranking include Gazebo (22 mentions), Webots (16), Bullet/PyBullet (16), CoppeliaSim/V-REP (13), MuJoCo (12), ODE (10), ROS 2 (10), and OpenRAVE (9). These counts help organize the candidate list and make the selection process auditable. The full ranked list with mention counts, including packages that did not survive the filtering step described in Section 3.5, is preserved in the project materials (`examples/software-list-counting-result.xlsx`, sheet `final.m`; companion source catalogue in `examples/sofware-counting-webs.csv`).

Mention count is a source-appearance signal and must be interpreted accordingly. It does not measure software quality, user adoption, research impact, maintainability, or suitability for any particular robotics task. A package may appear frequently because it is historically established, because it is broadly useful across application areas, or simply because the sources collected happened to surface it. A more specialized or newer package may be under-represented even when it is technically strong. The mention count is one input to candidate filtering, not the final ranking of the software.

3.5 Filtering Criteria

The ranked candidate list was filtered to identify packages suitable for repository-based State-of-the-Practice measurement. In the Domain X methodology, the initial filters are scope, usage, and age, applied in priority order until the list reaches a manageable size of approximately 30 packages Smith et al. [2021]. In this project, these filters are implemented through three criteria: open-source availability, repository-based measurability, and activity and maintenance status. Both the initial and filtered lists are preserved for traceability.

3.5.1 Open Source Availability

Open-source availability was required because the study depends on public software artifacts for measurement. The Domain X candidate criteria require that source code be viewable and express a preference for GitHub-style repositories from which empirical measures can be collected Smith et al. [2021]. Source code, repository history, issue trackers, documentation, and release information provide the evidence base for the measurements reported in Chapter 5.

This criterion led to the exclusion of several prominent robotics tools whose core components are not open source:

- **MATLAB and Simulink** are major platforms in both robotics research and industry, widely used for algorithm development, control design, and simulation. However, MATLAB is proprietary software. Its source code is not publicly available, and its development process is not accessible through public repositories. These properties prevent the kind of repository-based measurement that

this study depends on. Dr. Giamou confirmed that excluding MATLAB and Simulink does not misrepresent the domain.

- **Unity** is a widely used game engine with robotics simulation capabilities, but its core simulation engine is proprietary.
- **NVIDIA Isaac Sim** builds on NVIDIA Omniverse and provides advanced robotics simulation features, but its core platform components are not open source.
- **RaiSim** is a high-performance physics simulator for robotics, but its source code is not publicly available under an open-source license.

The exclusions above are methodological constraints, not judgements about the quality or importance of the excluded tools. Each of these platforms plays a significant role in robotics research and industry, and their exclusion limits the coverage of the study. Chapter 8 discusses this limitation further.

3.5.2 Repository-Based Measurability

Repository-based measurability requires that a package have sufficient public evidence to support the measurement process described in Section 3.7. Relevant evidence includes source repositories, commit history, issue data, documentation, release information, build or installation instructions, test artifacts, and project metadata. Packages without sufficient public evidence cannot be measured consistently and were excluded.

This criterion also governs how multi-repository projects are handled. Some

robotics software systems are distributed across multiple repositories or GitHub organizations. In such cases, the repositories most representative of the core software were selected for measurement, and this choice was documented as part of the measurement record. The same principle applies to projects that have changed names, migrated repositories, or spawned successor projects.

3.5.3 Activity and Maintenance Status

Activity and maintenance status were considered because the study focuses on contemporary software practice. In the Domain X methodology, age is one of the initial filtering criteria, measured by the last date when a change was made, with exceptions for older packages that remain highly recommended and currently in use Smith et al. [2021]. Tools that are no longer maintained provide weaker evidence about current development practices.

The primary activity indicator used in this study is the Domain X alive/dead rule: a package is considered alive if it has received commits or version releases within the last 18 months from the measurement date. The following packages were excluded on maintenance grounds:

- **MRDS** (Microsoft Robotics Developer Studio) has not received updates since 2012 and is no longer supported by Microsoft.
- **USARSim** (Unified System for Automation and Robot Simulation) is no longer actively maintained and does not represent current practice.

3.5.4 Scope and Coverage Filters

Two algorithm-level reference implementations were excluded to keep the selected packages methodologically coherent rather than for quality reasons. **GPMP2** (Gaussian Process Motion Planner 2) and **CHOMP** (Covariant Hamiltonian Optimization for Motion Planning) are optimization-based motion planning algorithms with public reference implementations. They are relevant to motion planning research but are narrower in scope than the general-purpose simulators, physics engines, planning frameworks, and middleware tools that dominate the candidate list. Including them alongside Gazebo, MoveIt!, or ROS 2 would force a comparison between full software stacks and single-algorithm libraries, which would distort the commonality analysis and the cross-package measurements. They were therefore excluded to maintain methodological consistency, not because of any quality concern.

Activity-related evidence is a snapshot. Repository activity, last commit dates, and release status can change after measurement. The measurement records in Chapter 5 include the measurement date to make the snapshot nature of the data explicit.

3.6 Final Software Set

After ranking by mention count and applying the filtering criteria described above, the selected set consists of 28 software packages. Table 3.1 lists the packages alphabetically with their primary roles in the robotics software ecosystem.

Table 3.1: Final software set with primary roles.

Software	**Primary Role**
AirSim	Simulator (aerial vehicles)
Bullet / PyBullet	Physics engine
CARLA	Simulator (autonomous driving)
CoppeliaSim (V-REP)	Simulator (general robotics)
DART	Dynamics library
Drake	Toolbox (dynamics, planning, control)
Flightmare	Simulator (quadrotor)
Gazebo	Simulator (general robotics)
GraspIt!	Manipulation (grasping)
Habitat	Simulator (embodied AI)
MORSE	Simulator (academic robotics)
MoveIt!	Manipulation (motion planning)
MRPT	Library (mobile robotics)
MuJoCo	Physics engine
Newton Dynamics	Physics engine
ODE (Open Dynamics Engine)	Physics engine
OMPL	Motion planning library
OpenRAVE	Planning framework
OpenRTM-aist	Middleware (RT components)
OROCOS	Control framework
PhysX (open-source SDK)	Physics engine
Pinocchio	Dynamics library
Project Chrono	Multi-physics simulation
ROS 2	Middleware and SDK
Simbody	Multibody dynamics library
SOFA	Simulation framework

This set is deliberately heterogeneous. Most entries are associated with simulation, physics, dynamics, planning, or robotics infrastructure, but they are not direct substitutes for one another. ROS 2, MoveIt!, and OMPL are boundary cases because they are not full simulators in the same sense as Gazebo or Webots. They are retained because the study scope is MPS software, not simulator applications alone. Chapter 4 provides a detailed analysis of software categories, commonalities, and variabilities across this set.

3.7 Measurement Procedure

The measurement procedure follows the Domain X method for collecting structured evidence about software packages in a selected domain Smith et al. [2021]. For each of the 28 selected packages, the study gathered source code, documentation, publications where applicable, repository metadata, issue-tracker evidence, release information, and installation evidence. The measurement record identifies the specific repository or repositories measured, since some robotics projects are distributed across multiple repositories.

3.7.1 Measurement Template

The primary measurement instrument is the Domain X Appendix B measurement template. The template contains approximately 108 fields when supplementary entries (free-text "additional comments" fields after each section and the GitHub-style and GitStats/SCC repository metric panels) are counted alongside the substantive questions. The substantive questions are grouped into the following sections:

1. **Summary Information** (17 questions): software name, URL, affiliation, purpose, number of developers, funding, release dates, status, license, platforms, software category, development model, publications, source code URL, programming languages, evidence of performance consideration, and additional comments.

2. **Installability** (15 questions): installation instructions (presence, single-document organization, linear ordering, completeness, and assumed pre-installed dependencies), compatible operating system versions, installation automation, error handling, installation validation procedure, number of installation steps, operating system used during measurement, dependencies (presence of guidance and required version listing), and uninstall behaviour (problems and residue).

3. **Correctness and Verifiability** (9 questions): requirements or theory documentation, correctness techniques, tutorial availability and quality, unit tests, and continuous integration evidence.

4. **Surface Reliability** (5 questions): installation and tutorial breakage, error messages, and recoverability.

5. **Surface Robustness** (2 questions): handling of unexpected input and edge cases.

6. **Surface Usability** (5 questions): tutorial, user manual, user characteristics, support model.

7. **Maintainability** (8 questions): version number, contribution guidance, available artifacts, issue tracking, issue closure rate, comment percentage, and version control.

8. **Reusability** (2 questions): number of code files and API documentation.

9. **Surface Understandability** (9 questions): code formatting, coding standards, identifier quality, constant usage, comment clarity, algorithm references, parameter ordering, and modularization.

10. **Visibility and Transparency** (4 questions): development process definition, process documentation, development environment documentation, and release notes.

Applying the same template to every package ensures that comparisons are based on a consistent set of observations rather than ad hoc impressions. Each quality section concludes with an overall impression score on a scale of 1 to 10, assigned using the scoring guidelines in the Domain X Appendix C impression calculator.

3.7.2 Repository Metrics

Repository metrics are collected separately from the quality impression scores. Three categories of repository data are gathered:

1. **GitHub-style metrics:** stars, forks, watchers, open pull requests, closed pull requests, number of developers, open issues, closed issues, initial release date, and last commit date.

2. **GitStats-style metrics:** number of text-based files, number of binary files, total lines, lines added, lines deleted, total commits, commits by year (last 5 years), and commits by month (last 12 months).

3. **SCC-style metrics:** number of text-based files, total lines, code lines, comment lines, and blank lines.

From these raw data, three processed measures are calculated:

1. **Alive/dead status:** alive is defined as the presence of commits or version releases within the last 18 months from the measurement date.

2. **Percentage of identified issues that are closed:** computed as closed issues divided by total identified issues.

3. **Percentage of code that is comments:** computed as comment lines divided by total lines.

3.7.3 Measurement Execution

The Domain X methodology recommends performing installation-based measurements in a controlled environment, typically a virtual machine, to ensure a clean and reproducible starting state Smith et al. [2021]. For this study, installation measurements were performed on a Linux-based virtual machine. The operating system, installation steps, dependencies, and any issues encountered were recorded for each package.

The methodology also recommends a time budget to keep the overall measurement workload feasible. The target is approximately 160 person-hours for the entire

domain, with 1 to 4 hours allocated per package for template completion and up to 2 hours for installation Smith et al. [2021]. Measurement was conducted during May 2026. All time-sensitive data (last commit dates, issue counts, stars, forks, etc.) should be understood as snapshots taken during this period.

Several measurement questions require explicit interpretation rules:

- **Last commit date** is recorded at the time of measurement and may change for active repositories.
- **Percentage of identified issues that are closed** is computed consistently as closed issues divided by total identified issues, using GitHub issue data where available.
- **Evidence that performance was considered** is assessed by searching software artifacts for explicit references to speed, storage, throughput, performance optimization, parallelism, multi-core processing, or similar considerations.
- **Percentage of code that is comments** is computed using the SCC tool's line-based method, as comment lines divided by total lines.

3.7.4 Ranking with AHP

After the quality measures are collected, the Domain X methodology uses the Analytic Hierarchy Process (AHP) to support ranking across the nine quality categories Saaty [1980]. AHP is a decision-making technique that uses pairwise comparisons between alternatives to calculate relative scores. It organizes multiple criteria in a hierarchical structure and enables the combination of individual quality rankings into an overall ranking based on the relative priorities assigned to each quality.

In this study, AHP would be applied to the nine quality impression scores for the 28 software packages and would produce a ranking that is an analysis aid for comparing evidence under the chosen criteria, not a universal claim that one package is best for every robotics use case. Sensitivity analysis would then assess whether the ranking is stable under reasonable variations in quality weightings. AHP rankings are not computed in this report, because a defensible criterion-weighting elicitation has not been completed at the time of writing; Chapter 5 therefore reports the per-package quality-mean ranking and a comparison with community indicators in their place.

3.8 Commonality Analysis Procedure

The commonality analysis identifies shared and variable capabilities across the selected software packages, treating them as members of a software family as defined by Parnas [1976] and Weiss [1997]. The analysis supplements repository-based metrics by describing what the packages do and how their roles differ, providing context for interpreting the measurement results reported in Chapter 5.

Following the Domain X methodology, the analysis proceeds in six steps:

1. **Write an introduction** to the domain and the software family.

2. **Write an overview of domain knowledge,** summarizing the key concepts and terminology relevant to MPS software.

3. **List commonalities:** identify goals, assumptions, functions, artifacts, and domain concepts that recur across multiple family members. For example, many of

the selected packages share the goal of simulating robot dynamics, the assumption that robot models are described using standard formats (such as URDF or SDF), and the function of providing an API for programmatic control.

4. **List variabilities:** identify dimensions on which family members differ. These include software role (simulator, physics engine, planning library, middleware), supported platforms, licensing, programming language interfaces, documentation style, and intended use cases.

5. **List parameters of variation:** for each variability, describe the possible values. For example, the software role parameter can take values such as general-purpose simulator, domain-specific simulator, physics engine, dynamics library, motion planning library, manipulation framework, or middleware.

6. **Add terminology, definitions, and acronyms** that are standard in the domain, to ensure that the analysis is accessible to readers outside robotics.

The procedure first groups the selected packages into broad categories based on their primary roles: robotics simulators, physics and dynamics engines, and planning or middleware tools. It then identifies capabilities that appear across multiple categories, such as support for robot model representation, simulated environments, motion planning workflows, dynamics computation, and integration with robotics ecosystems.

Because the selected set is heterogeneous, the commonality analysis does not force all tools into a single comparison template. Its purpose is to describe shared concerns and meaningful differences, not to claim that every package provides identical functionality. The analysis is based on checked evidence from documentation, repositories,

publications, and other reliable project materials. Capabilities that appear likely but have not been verified remain marked as needing confirmation.

3.9 Interview Component

The Domain X methodology includes a developer survey component designed to capture information that is not visible in public repositories: the development process as experienced by practitioners, the pain points they encounter, their perceptions of software quality threats, and the strategies they use to address those threats Smith et al. [2021]. The methodology prescribes semi-structured interviews based on a list of 20 questions covering developer background, project organization, user understanding, development obstacles (past and present), documentation practices, and specific software qualities including maintainability, understandability, usability, and reproducibility.

For this study, the interview component is associated with research ethics approval through the McMaster Research Ethics Board (MacREM). The project materials indicate that interviews with software developers, maintainers, or programmers associated with the studied packages may form part of the broader State-of-the-Practice exercise.

As of the current report draft, approved interview data (including transcripts, participant responses, approved summaries, or final interview findings) is not yet available. The interview component is therefore described as planned or pending. Chapter 7 is reserved for this material and currently describes the planned protocol, recruitment strategy, and analysis framework. Interview findings will be added only when supported by approved data.

The methodology recommends sending interview requests to developers from all packages in the study set to avoid selection bias. Prior applications of the methodology report response rates roughly in the 15%–30% range Smith et al. [2021, 2024a]; the exact rate for this study will depend on outreach timing and the response practices of the contacted developers. Interview contacts are typically identified through project websites, code repositories, publications, or institutional biography pages.

Chapter 4

Selected Software and Commonality Analysis

This chapter presents the 28 software packages selected for the study and analyzes them as members of a software family. It completes the answer to RQ1 by characterizing the selected set, and provides the commonality context needed to interpret the per-quality scores reported for RQ2 in Chapter 5. Section 4.1 provides an overview of the selected set. Section 4.2 groups the packages into three broad categories based on their primary roles. Section 4.3 identifies common capabilities shared across the family. Section 4.4 identifies dimensions on which the packages differ. Section 4.5 describes the parameters of variation and their binding times. Section 4.6 discusses boundary cases that occupy intermediate positions in the classification.

The commonality analysis follows the approach described by Weiss [1997] and is based on evidence from project documentation, repositories, publications, and other reliable sources. Capabilities described as likely but not yet verified against project materials are explicitly noted.

4.1 Overview of Selected Software

The 28 software packages selected through the discovery and filtering process described in Chapter 3 represent a cross-section of the robotics software ecosystem for motion planning and simulation (MPS software). The set includes tools developed in academic research groups (such as Drake, DART, and OMPL), tools stewarded by open-source organizations and consortia (ROS 2 and Gazebo through Open Robotics, which together also coordinate with the ROS-Industrial Consortium), tools released by corporate research labs (MuJoCo by Google DeepMind, AirSim by Microsoft Research, and Habitat by Meta AI Research), and tools maintained by smaller communities or individual contributors (such as ODE and Newton Dynamics).

The packages span a period of active development from 2001 (ODE's initial release) to the present, with most showing recent repository activity as of the May 2026 measurement date. All 28 packages have public source repositories. Twenty-seven use standard open-source licenses; CoppeliaSim follows a mixed model that combines a GPL-licensed open-source core with commercial and educational distributions. BSD is the most common license (11 packages), followed by Apache 2.0 (5) and MIT (4); the remainder use LGPL, GPL, zlib, or mixed/dual schemes, as detailed in Table 5.1 in Chapter 5. The majority of packages are written primarily in C++ with Python bindings, reflecting the dominant languages in the robotics software community. Twenty-seven packages support Linux (PhysX's open-source SDK listed only Windows as a target in the measured snapshot). Most packages also support Windows or macOS, with cross-platform support varying by project (see Chapter 5 for details).

This chapter does not report measurement scores or quality rankings; those appear

in Chapter 5. Its purpose is to describe what the selected packages do, how they relate to one another, and what dimensions of variation characterize the software family, so that the measurement results in Chapter 5 can be interpreted in context.

4.2 Software Categories

The selected packages can be grouped into three broad categories based on their primary function within a robotics workflow. These categories are not strict boundaries (some packages span multiple roles) but they provide a useful structure for the commonality analysis.

4.2.1 Robotics Simulators

Robotics simulators provide virtual environments for representing robots, worlds, sensors, and actuators. They enable researchers and engineers to test algorithms, validate designs, and collect synthetic data before deploying to physical hardware. Packages in this category typically offer graphical rendering, physics simulation, sensor models, and programmatic interfaces for controlling simulated robots.

The following packages from the selected set are primarily classified as robotics simulators:

- **Gazebo** (including the Ignition/Gazebo Sim lineage) is a long-established open-source 3D robotics simulator developed by Open Robotics. The Gazebo and ROS intellectual property remains with the Open Source Robotics Foundation; the commercial subsidiary (OSRC) was acquired by Intrinsic Innovation, an Alphabet company, in December 2022, and the development team moved with it.

Gazebo provides high-fidelity physics (using multiple physics engine backends including ODE, Bullet, DART, and Simbody), rendering, and sensor models, with deep integration into the ROS ecosystem.

- **Webots** is a multi-platform robot simulator originally developed at EPFL and now maintained by Cyberbotics. It supports modeling, programming, and simulating robots, vehicles, and mechanical systems, with a large library of pre-built robot models.

- **CoppeliaSim** (formerly V-REP) is a versatile robot simulation framework developed by Coppelia Robotics. It supports rapid algorithm development, factory automation simulation, prototyping, verification, and digital twin applications.

- **AirSim** is an open-source simulator for autonomous vehicles (drones and ground vehicles) built on Unreal Engine, originally developed by Microsoft Research. The original `microsoft/AirSim` repository was archived in 2022 in favour of a successor project (Project AirSim); the measurement record refers to that archived repository, and community forks continue to extend it.

- **CARLA** is an open-source autonomous driving simulator built on Unreal Engine, developed by the Computer Vision Center in Barcelona in collaboration with Intel Labs and the Toyota Research Institute. It provides photorealistic urban environments and a broad sensor suite (cameras, LiDAR, semantic LiDAR, depth sensors, GNSS, and other configurable sensors) for autonomous driving research.

- **MORSE** (Modular OpenRobots Simulation Engine) is a generic academic

robotic simulator based on Blender and the Bullet physics engine, emphasizing middleware-agnostic design compatible with ROS, YARP, and other frameworks.

- **Habitat** is a high-performance 3D simulator for embodied AI research developed by Meta AI Research. It enables training and evaluation of embodied AI agents in photorealistic indoor environments.

- **Flightmare** is a flexible modular quadrotor simulator developed by the Robotics and Perception Group at the University of Zurich and ETH Zurich. It separates rendering from the RL environment for flexible configuration between fast and photorealistic modes.

Several additional packages provide simulation capabilities as part of broader functionality. Drake includes simulation features within its model-based design toolbox. MRPT includes a 3D simulation environment alongside its mobile robotics libraries. SOFA provides interactive simulation for medical and soft robotics applications. These packages may be considered simulators in some contexts but serve broader roles.

4.2.2 Physics Engines and Dynamics Libraries

Physics engines and dynamics libraries provide the computational foundation for simulating rigid-body and multibody dynamics, collision detection, and contact resolution. They are often used as backends by higher-level simulators but can also be used directly by researchers and developers.

The following packages are primarily classified as physics engines or dynamics libraries:

- **Bullet / PyBullet** is a widely used open-source physics engine for simulation, robotics, and deep reinforcement learning. PyBullet provides Python bindings that have made it popular in the machine learning community.
- **MuJoCo** (Multi-Joint dynamics with Contact) is a general-purpose physics simulator originally developed by Emo Todorov at the University of Washington and commercialized through Roboti LLC. Google DeepMind acquired the codebase in October 2021, released it as open source under Apache 2.0 in May 2022, and continues to maintain it. MuJoCo is designed for fast and accurate simulation of articulated structures and is widely used in robotics and reinforcement learning research.
- **ODE** (Open Dynamics Engine) is one of the earliest open-source libraries for simulating rigid body dynamics. It has been used extensively in robotics and served as a physics backend for Gazebo Classic.
- **PhysX** is NVIDIA's high-performance real-time physics engine SDK. The open-source SDK provides capabilities for simulating rigid bodies, particles, vehicles, and deformable bodies, primarily targeted at real-time simulation applications.
- **DART** (Dynamic Animation and Robotics Toolkit) is a C++ physics engine and kinematics/dynamics library developed at Georgia Tech and other institutions. It emphasizes accurate and stable simulation for manipulation and locomotion research.

- **Simbody** is a high-performance multibody dynamics library developed at Stanford University for simulating articulated biomechanical and mechanical systems. It is the physics engine underlying OpenSim (biomedical simulation software).

- **Pinocchio** is a fast and flexible C++ library for rigid-body dynamics computations and their analytical derivatives, developed primarily at Inria and LAAS-CNRS. It implements state-of-the-art algorithms for kinematics, dynamics, and Jacobians.

- **Project Chrono** is a high-performance open-source multi-physics simulation library developed at the University of Wisconsin-Madison and the University of Parma. It supports multibody dynamics, finite element analysis, vehicle dynamics, and GPU-accelerated simulation.

- **Newton Dynamics**, developed by Julio Jerez and Alain Suero, is a cross-platform physics simulation engine optimised for real-time deterministic rigid-body simulation. It is used in game development and robotics simulation.

- **SOFA** (Simulation Open Framework Architecture) is an open-source real-time simulation framework developed primarily at Inria. It focuses on deformable object simulation, finite element methods, and haptic feedback, with applications in surgical simulation and soft robotics.

4.2.3 Motion Planning, Middleware, and Manipulation Tools

Motion planning libraries, robotics middleware, and manipulation tools support planning, integration, communication, grasping, and robotic system development. These

packages are not simulators in the same sense as Gazebo or Webots, but they belong to the robotics workflows that combine planning, simulation, and execution. GraspIt!, included here under manipulation, is consistent with the role assigned in Chapter 3, Table 3.1.

The following packages are primarily classified as planning or middleware tools:

- **ROS 2** (Robot Operating System 2) is an open-source software development kit and middleware framework for building robot applications. It provides communication infrastructure (based on DDS), hardware abstraction, drivers, libraries, visualization tools, and a large ecosystem of community packages. ROS 2 is the successor to ROS 1 and represents a major redesign with emphasis on real-time control, multi-robot deployment, and production use.

- **MoveIt!** (referring to the active MoveIt 2 generation for ROS 2 in this study) is an open-source robotics manipulation framework. MoveIt originated at Willow Garage in 2011 (Sachin Chitta, Ioan Sucan, and others) and was later supported by SRI International (2013–2015); PickNik Robotics has led MoveIt and MoveIt 2 since Willow Garage shut down, with ROS-Industrial Consortium support through the ROSIN project. MoveIt 1 for ROS 1 is end-of-life, and the measurements in this report refer to MoveIt 2. It provides motion planning, kinematics, collision checking, trajectory execution, and perception integration for robot arms and manipulators, integrating OMPL and other planners as plugins.

- **OMPL** (Open Motion Planning Library) is an open-source C++ library for sampling-based motion planning algorithms developed at Rice University. It

provides implementations of widely used planners such as PRM, RRT, RRT*, and EST, and is the default planning backend for MoveIt!.

- **OpenRAVE** (Open Robotics Automation Virtual Environment) is an open-source robotics planning framework originally developed by Rosen Diankov during his PhD at Carnegie Mellon University and subsequently maintained at the JSK Robotics Lab, University of Tokyo, where Diankov was a postdoc before founding MUJIN Inc. in Japan. It focuses on simulation and analysis of kinematic and geometric information for motion planning, particularly for manipulation tasks.

- **Drake** is an open-source C++ and Python toolbox for model-based design and verification of robotics systems, developed at MIT and the Toyota Research Institute. It covers multibody dynamics, trajectory optimization, model predictive control, perception, and manipulation planning.

- **OROCOS** (Open RObot COntrol Software) is an open-source C++ framework for real-time robot control developed at KU Leuven. Its major components include the Real-Time Toolkit for component-based hard real-time software and the Kinematics and Dynamics Library.

- **MRPT** (Mobile Robot Programming Toolkit) is an open-source cross-platform C++ library for mobile robotics research, providing algorithms for SLAM, computer vision, motion planning, and sensor data processing.

- **YARP** (Yet Another Robot Platform) is open-source middleware for humanoid and general robotics, developed at the Istituto Italiano di Tecnologia. It provides communication infrastructure for distributed robot systems and is the

primary middleware for the iCub humanoid robot platform.

- **OpenRTM-aist** is an implementation of the OMG-standardized Robot Technology Component standard, developed at AIST in Japan. It provides a component-based framework for creating and connecting robot software modules.

- **GraspIt!** is a simulation environment for robotic grasping research developed at Columbia University. It supports loading robot hand models and 3D object models to plan, evaluate, and visualize grasps.

4.3 Commonalities

Despite the diversity of the selected packages, several capabilities and concerns recur across the software family. These commonalities are organized around the abstraction of a typical robotics software workflow: model representation, computation, interaction, and documentation.

C1: Robot Model Representation. All selected packages provide or consume some form of robot model representation. Simulators and physics engines represent robot kinematics and dynamics through internal model structures (often supporting standard formats such as URDF, SDF, or MJCF). Motion planning libraries operate on kinematic and geometric descriptions of robots and environments. Middleware packages define standardized message formats for communicating robot state.

C2: Physics or Kinematics Computation. The packages in the simulator and physics engine categories share the capability of computing robot motion under physical constraints. This includes forward and inverse kinematics, rigid-body dynamics, collision detection, and contact resolution. Even middleware and planning

tools typically include kinematic computation capabilities or depend on libraries that provide them.

C3: Programmatic Interface. All 28 packages provide a programmatic interface (typically a C++ API, often with Python bindings) that allows developers to control the software, configure simulations or planners, and integrate the package into larger robotics workflows.

C4: Open-Source Repository Hosting. All 28 packages use public version control hosting (GitHub for 27 of 28 packages) with visible commit history, issue tracking, and pull request management. This shared infrastructure is both a selection criterion and a commonality of the selected set.

C5: Documentation. All packages provide some form of user-facing documentation, typically including installation instructions, a getting-started tutorial, and API reference material. The depth, currency, and completeness of this documentation vary considerably, as discussed in Chapter 5.

C6: Integration with Robotics Ecosystems. Most packages provide or support integration with broader robotics ecosystems. ROS 2 integration is the most common pattern, with many simulators offering ROS 2 interfaces for publishing sensor data and accepting control commands. Some packages (such as Gazebo and MoveIt!) are deeply embedded in the ROS ecosystem, while others maintain their own integration frameworks.

C7: Simulation or Planning Loop. Simulators and planners share a common architectural pattern: a loop that iteratively updates the system state. In simulators, this is the physics stepping loop that advances the simulation in time. In planners, it is the search or optimization loop that explores the configuration space to find a

valid path.

4.4 Variabilities

The selected packages differ along several important dimensions. These variabilities are not deficiencies; they reflect the different roles that packages play within the robotics software ecosystem.

V1: Primary Software Role. The most fundamental variability is the role a package plays in a robotics workflow. As described in Section 4.2, packages range from full simulation environments (Gazebo, Webots, CoppeliaSim) to physics engines (Bullet, MuJoCo, ODE) to planning libraries (OMPL) to middleware frameworks (ROS 2, YARP).

V2: User Interface Model. Packages differ in how users interact with them. Some provide graphical user interfaces with 3D rendering, scene editors, and interactive controls (Gazebo, Webots, CoppeliaSim). Others are primarily libraries or APIs intended for programmatic use (OMPL, Pinocchio, ODE). Some provide both (Drake, MRPT).

V3: Domain Specificity. Some packages are general-purpose robotics simulators or engines (Gazebo, Bullet, MuJoCo), while others target specific application domains: CARLA and AirSim for autonomous vehicles, Flightmare for quadrotor flight, Habitat for embodied AI in indoor environments, SOFA for medical simulation, and GraspIt! for robotic grasping.

V4: Physics Fidelity and Approach. Physics engines differ in their simulation approach and fidelity. Some prioritize speed for real-time or reinforcement learning applications (MuJoCo, Bullet), while others emphasize physical accuracy for

engineering analysis (Project Chrono, Simbody). Differences include the choice of constraint solver, collision detection algorithm, integration method, and support for deformable bodies or fluid dynamics.

V5: Platform Support. While all packages support Linux, support for other platforms varies. Some packages provide first-class support for Windows and macOS (Webots, CoppeliaSim, Bullet), while others are primarily developed for Linux with limited cross-platform support.

V6: Programming Language Interfaces. C++ is the dominant implementation language across the selected packages, but packages differ in the range and quality of language bindings they provide. Python bindings are increasingly common because Python has become central to robotics research and machine learning. Fewer packages provide interfaces for languages such as MATLAB, Java, or Rust.

V7: ROS Integration Depth. Packages vary in their relationship to the ROS ecosystem. Some (Gazebo, MoveIt!) are tightly integrated and depend on ROS for core functionality. Others (Webots, CARLA) provide ROS bridges as optional components. Some (YARP, OROCOS) are alternatives to ROS that provide their own middleware infrastructure.

V8: License Model. Although all packages are open source, they use different licenses. The most common are Apache 2.0, BSD (2-clause and 3-clause), MIT, and LGPL. License choice affects how the software can be reused and integrated into other projects.

V9: Community and Governance. Packages differ in their development governance. Some are stewarded by established organizations (Open Robotics for Gazebo and ROS 2, Google DeepMind for MuJoCo, NVIDIA for PhysX), some by

academic research groups (Drake at MIT/TRI, OMPL at Rice, DART at Georgia Tech), and some by individual maintainers or small communities (ODE, Newton Dynamics, GraspIt!).

4.5 Parameters of Variation

For each variability identified in Section 4.4, Table 4.1 summarizes the parameters of variation and their typical binding times. Binding time indicates when a particular choice is fixed: at specification time (when the software is designed), build time (when it is compiled), configuration time (when it is set up), or run time (during execution).

Table 4.1: Parameters of variation and binding times.

Variability	**Parameters of Variation**	**Binding Time**
V1: Primary role	Simulator, physics engine, dynamics library, motion planning library, manipulation framework, middleware, multi-purpose toolbox	Specification time
V2: User interface	Graphical (3D scene, editor), programmatic (API only), both	Specification time
V3: Domain specificity	General-purpose robotics, autonomous driving, aerial vehicles, embodied AI, medical/soft robotics, grasping, mobile robotics	Specification time
V4: Physics fidelity	Real-time (game-quality), high-fidelity (engineering-quality), configurable trade-off	Specification / configuration time
V5: Platform support	Linux-only, Linux + Windows + macOS, web-based	Specification / build time
V6: Language bindings	C++ only, C++ with Python, multi-language (Python, MATLAB, Java, etc.)	Build / configuration time
V7: ROS integration	Native (ROS-dependent), bridge (optional ROS interface), independent (separate middleware), alternative (non-ROS middleware)	Configuration time
V8: License	Permissive (BSD, MIT, Apache 2.0), reciprocal (LGPL, GPL)	Specification time
V9: Governance	Single institution or company,	Specification

The binding times reflect when a user or integrator can make choices. A package's primary role and governance model are fixed at specification time—they are inherent to the software's design. Platform support is largely determined at build time, though some packages allow configuration-time choices. ROS integration depth can often be chosen at configuration time, as many packages offer optional ROS bridges. Physics fidelity involves trade-offs that may be configured at both specification time (choice of engine) and run time (choice of solver parameters or time step).

4.6 Boundary Cases

Several of the selected packages occupy intermediate positions in the classification presented above. These boundary cases are not classification errors; they illustrate the interconnected nature of the robotics software ecosystem and the difficulty of drawing sharp boundaries between categories.

ROS 2 is classified as middleware in this analysis, but its scope extends well beyond communication infrastructure. ROS 2 provides an SDK with libraries, drivers, visualization tools (RViz), simulation integration (through Gazebo), and a package management system. It is the de facto integration layer for robotics software components, and many other packages in the selected set assume it as their runtime environment. Its inclusion in a study of MPS software is justified by its central role in the workflows that combine these capabilities.

Drake spans multiple categories. It provides simulation capabilities without being a full simulator in the sense of Gazebo or Webots. It includes sophisticated multibody dynamics without being solely a physics engine. It offers trajectory optimization and model predictive control without being only a planning library. Drake is best

understood as a model-based design toolbox that integrates dynamics, planning, and control into a unified framework.

MoveIt! and **OMPL** are closely related but serve different roles. OMPL is a general-purpose motion planning library that can be used independently. MoveIt! is a manipulation framework that integrates OMPL (among other planners) with kinematics, collision checking, and trajectory execution for robot arms. OMPL is classified as a motion planning library and MoveIt! as a manipulation framework, but the boundary between them reflects architectural layering rather than functional separation.

SOFA is classified as a simulation framework but differs from general-purpose robotics simulators in its focus on deformable objects and finite element methods. Its primary application domain is surgical simulation and medical education, making it a domain-specific simulator rather than a general robotics tool.

These boundary cases are not treated as measurement anomalies. In the chapters that follow, measurements are reported for each package individually, and comparisons are interpreted in the context of each package's role within the robotics software ecosystem.

Chapter 5

Ranking Projects by Quality Measure

This chapter answers RQ2 by ranking the 28 selected software packages on the nine Domain X quality categories and by comparing that ranking with community-facing indicators of visibility and adoption. All data was collected during May 2026 and represents a snapshot of repository state, documentation, and project activity at that time. The quality measurements follow the Domain X Appendix B template and cover the nine quality categories described in Section 2.4. Section 5.1 presents summary metadata. Sections 5.2 through 5.10 report per-category results. Section 5.11 reports repository metrics. Section 5.12 synthesizes the per-category findings and reports the quality-mean ranking. Section 5.13 compares the quality-mean ranking with community indicators of visibility and adoption.

5.1 Repository and Project Metadata

Table 5.1 presents summary metadata for all 28 packages, including the primary development organization, license, initial release date, and primary programming languages. Platform support is summarized in the narrative following the table. The packages are listed alphabetically.

Table 5.1: Summary metadata for the 28 selected software packages (May 2026). The Initial Release column records the date associated with the measured public repository: for packages with a single continuous open-source history this is the first public release, while for packages that were renamed, re-licensed, or migrated to open source after an earlier proprietary or pre-release phase (for example CoppeliaSim, MuJoCo, Webots, Gazebo, PhysX), the date reflects that later event rather than the project's original creation.

Software	**Primary Affiliation**	**License**	**Initial Release**	**Primary Languages**
AirSim	Microsoft Research	MIT	2017	C++, Python
Bullet/PyBullet	Community	zlib	2006	C++, Python
CARLA	CVC Barcelona, Intel	MIT	2017	C++, Python
CoppeliaSim	Coppelia Robotics	GPL + comm.	2019	C++, Python
DART	Georgia Tech	BSD	2011	C++, Python
Drake	TRI, MIT	BSD	2010	C++, Python
Flightmare	U. Zurich, ETH	MIT	2020	C++, Python
Gazebo	Open Robotics	Apache-2.0	2014	C++, Python
GraspIt!	Columbia U.	GPL v3+	2010	C++, Python
Habitat	Meta AI Research	MIT	2019	C++, Python
MORSE	LAAS-CNRS	BSD	2009	C, Python

All 28 packages have publicly accessible source repositories. Twenty-seven follow an open-source development model under standard permissive or copyleft licenses; CoppeliaSim is the exception, with a mixed model that combines a GPL-licensed open-source codebase with commercial and educational licenses for its packaged distribution. The most common licenses are BSD (11 packages), Apache 2.0 (5), and MIT (4). The packages span more than two decades of development history: ODE is the oldest, with its initial release in 2001, while PhysX's open-source SDK was released in 2022.

C++ is the dominant implementation language, listed as a primary or major language by 25 of 28 packages in the measurement records. Three packages fall outside that count: MORSE (primary languages: C and Python), ODE (primary language listed as "unclear" because the public source tree mixes C and C++ files without a single declared primary), and ROS 2 (also "unclear" because the ROS 2 organization spans hundreds of repositories with different primary languages, although the core communication libraries themselves are in C++). Python is a primary or secondary language in most packages, reflecting its growing role in robotics research and its use for scripting, bindings, and machine learning integration. Twenty-seven packages support Linux. The recorded exception is PhysX, where the measurement metadata captured only Windows as a target platform; the public NVIDIA-Omniverse/PhysX repository in fact provides a Linux build path tested on Ubuntu 20.04 and later. The platform field for PhysX therefore under-reports its actual platform coverage and should be read alongside that documentation. Twenty-four packages support Windows and 23 support macOS.

As of the May 2026 measurement date, 24 of the 28 packages (86%) were classified as alive under the Domain X 18-month activity rule, which defines alive as the presence of any commit or release in the preceding 18 months. Four packages were classified as dead: OpenRAVE (last commit August 2024), MORSE (last commit March 2022), GraspIt! (last commit July 2020), and Flightmare (last commit May 2023). ODE is technically alive under the 18-month rule (last commit 2026-04-14) but its commit activity in the trailing twelve months is sparse (five commits across the year); the analysis that follows treats ODE as a low-activity alive package and discusses its limitations alongside the dead packages where appropriate.

The number of developers (anyone who has made at least one accepted commit) ranges from 6 (CoppeliaSim, Flightmare) to 598 (MoveIt!). The median is approximately 128.5 (the mean of the 14th and 15th values, YARP at 125 and MRPT at 132). The total number of commits across all measured repositories exceeds 240,000. Drake leads with 34,786 commits, followed by SOFA (27,404) and Project Chrono (19,510). These repository-level metrics are snapshots and should not be interpreted as direct measures of project scale or quality without considering factors such as repository age, commit granularity, and whether development activity is concentrated in a single repository.

PhysX requires a particular caveat. The measured repository (`NVIDIA-Omniverse/PhysX`) is the public open-source SDK release of NVIDIA's PhysX engine; the broader production engine and its associated engineering process are developed in an internal NVIDIA codebase that is not publicly accessible. All PhysX scores reported in this chapter, including the small public commit count, the platform support recorded as Windows only, and the per-quality impression scores, reflect only the public SDK

surface as it appeared at the measurement date. They should not be read as an assessment of the PhysX engine as a product, or of NVIDIA's internal development practices. Where PhysX scores appear lower than peers in subsequent sections, the gap reflects what is publicly observable rather than a verdict on the engine itself.

5.2 Installability

Installability measures the effort required to install and uninstall software in a specified environment. All installation measurements were performed on Linux-based virtual machines. Table 5.2 summarizes key installability indicators across the selected packages.

Table 5.2: Key installability indicators across 28 packages.

Indicator	**Packages**	**%**
Installation instructions present	28	100%
Instructions in a single document or page	26	93%
Instructions are linear (single series of steps)	26	93%
Instructions assume no pre-installed dependencies	22	79%
Compatible OS versions listed	17	61%
Installation automation provided (script, makefile, etc.)	28	100%
Installation validation procedure available	27	96%
Dependency installation instructions provided	21	75%
Required package versions listed	19	68%
No problems caused by uninstall (where available)	27	96%

All 28 packages provide installation instructions, and all provide some form of installation automation (makefiles, scripts, package managers, or container definitions). For comparison, the Medical Imaging study Smith et al. [2024a] reported that only about half of MI packages installed successfully, so the MPS coverage on this dimension is markedly better.

The number of installation steps ranged from 1 to 11, with a median of 2. The number of extra software packages required before or during installation ranged from 0 to 10, with a median of 1. GraspIt! required the most manual effort (11 steps, 10 extra packages), consistent with its dead status and lack of recent maintenance. At the other extreme, ten packages (including Webots, MuJoCo, DART, and Simbody) needed no extra packages at all, and nine could be installed in a single step using package managers or pre-built binaries (MuJoCo, Simbody, and MORSE achieved both).

Installation broke for only 2 of the 28 packages during measurement (OpenRAVE and YARP; OpenRAVE is also classified as inactive in Section 5.1). In both cases the installation instance was recoverable. No package broke during initial tutorial testing.

The overall installability impression scores (on a 1–10 scale) range from 4 to 10, with a mean of 7.4. The top-scoring packages are Gazebo (10), Pinocchio (10), Bullet/PyBullet (9), MORSE (9), and OMPL (9). The lowest-scoring packages are ODE (4), PhysX (4), and GraspIt! (5). For ODE, the low score reflects outdated installation documentation (the official guide dates from 2006 and refers to GNU make, VC6 project files, and manual copying of library files). For PhysX, the low score reflects difficulty locating the correct build documentation.

5.3 Correctness and Verifiability

Correctness and verifiability assess whether a program matches its specification and the ease with which correctness can be checked. Table 5.3 summarizes key indicators.

Table 5.3: Correctness and verifiability indicators.

Indicator	**Packages**	**%**
Reference to requirements specifications or theory manuals	26	93%
Getting-started tutorial present	28	100%
Tutorial instructions are linear	27	96%
Tutorial provides expected output	25	89%
Tutorial output matches expected output	23	82%
Unit tests available	27	96%
Evidence that performance was considered	28	100%

All 28 packages use at least one technique to build confidence in correctness. Unit tests, as recorded in Table 5.3, are present for 27 of 28 packages (96%); of those, 26 also have an automated-testing setup (CI runs that exercise the tests on each push or pull request), giving an automated-testing rate of 26 of 28 (93%). Assertions in code are used by 23 of 28 packages (82%). Documentation generation tools such as Doxygen or Sphinx are used by 12 packages (43%). Ten packages combine all three techniques: DART, Drake, Habitat, Newton Dynamics, OMPL, OROCOS, PhysX, Pinocchio, Project Chrono, and SOFA all use automated testing, assertions, and a documentation generator together.

All 28 packages provide a getting-started tutorial, and 27 of 28 provide linear tutorial instructions. Twenty-five of 28 packages publish an expected tutorial output; the remaining three (SOFA, Flightmare, Newton Dynamics) do not, so for those packages the match check is not applicable. Of the 25 with an expected output, 23 matched on the measurement run (23 of 25 = 92%) and two more (MuJoCo and OpenRTM-aist) were recorded as not applicable for the match check because the tutorial output depended on environment-specific configuration that the standard measurement procedure could not reproduce. The match indicator in Table 5.3 therefore reports 23 of 28 (82%); five packages fall outside "output matches expected output" under any reading.

Unit tests are available for 27 of 28 packages. CoppeliaSim's unit test status is unclear because its test infrastructure is not publicly visible in the same way as the other packages, a consequence of its mixed commercial/open-source licensing model.

The overall correctness and verifiability impression scores range from 5 to 10, with a mean of 7.7. The top-scoring packages are Gazebo (10), Drake (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9). The lowest-scoring are GraspIt! (5) and Newton Dynamics (5).

5.4 Surface Reliability

Surface reliability assesses failure-free operation during the limited interactions performed during measurement: installation and initial tutorial use. These are surface checks, not comprehensive reliability experiments.

As noted in Section 5.2, installation broke for 2 of 28 packages (OpenRAVE and YARP). In both cases, a descriptive error message was displayed and the installation

instance was recoverable. No package broke during initial tutorial testing.

The overall surface reliability impression scores range from 6 to 10, with a mean of 7.8. The top-scoring packages are Gazebo (10), Drake (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9). Three packages scored below 7: PhysX (6), GraspIt! (6), and Newton Dynamics (6).

The generally high surface reliability scores reflect the fact that most packages installed without incident and all tutorials ran without breakage. This is a positive signal for the domain, though it should be interpreted cautiously: the measurement procedure involves a shallow interaction (installation plus one tutorial) on a single operating system, and does not constitute a systematic reliability evaluation.

5.5 Surface Robustness

Surface robustness assesses whether software handles unexpected or unanticipated input reasonably, including edge cases such as malformed input files. All 28 packages were assessed as handling unexpected input reasonably (producing appropriate error messages or graceful degradation rather than crashes or undefined behaviour). For the newline-conversion check (replacing newlines with carriage returns and newlines in plain-text input files), 25 of 28 packages handled the conversion gracefully; the remaining three (CoppeliaSim, ODE, PhysX) were recorded as not applicable because their workflows do not take plain-text input files in this form.

The overall surface robustness impression scores range from 5 to 10, with a mean of 7.4. Gazebo scores highest (10), followed by Drake (9). The lowest-scoring package is GraspIt! (5). The narrow range of scores reflects the binary nature of the surface robustness checks in the Domain X template: most packages either pass both checks

or pass one of two, resulting in similar scores.

5.6 Surface Usability

Surface usability assesses the presence and quality of user-facing documentation, tutorials, and support infrastructure. It is not a full usability evaluation; no user studies or task-completion experiments were conducted.

All 28 packages provide a getting-started tutorial and a user manual. The Medical Imaging study Smith et al. [2024a], by comparison, found that only 22 of 29 MI packages (76%) provided a user manual. Only one package in the current study (MuJoCo) documents expected user characteristics, which is consistent with findings from other domains where this practice is rare Smith et al. [2024a,b].

Table 5.4 summarizes the user support models adopted by the study packages. Most packages use multiple support channels.

Table 5.4: User support models across 28 packages.

Support Channel	**Packages Using**
GitHub Issues	28
GitHub Discussions	10
Official forum or discussion board	9
Mailing list / Google Group	8
Discord server	6
Documentation website or wiki	28
Stack Exchange / Stack Overflow tag	5
ROS Answers integration	4

GitHub Issues is the universal support channel, used by all 28 packages, and a dedicated documentation website or wiki is also present for every package. The other channels show more variation: 10 packages use GitHub Discussions, 9 maintain an official forum, 8 use a mailing list or Google Group, 6 maintain a Discord server, 5 use a Stack Exchange or Stack Overflow tag, and 4 integrate with ROS Answers. This variation partly reflects project age and governance model. Newer packages and those stewarded by companies or large labs tend to favour Discord and GitHub Discussions, while older academic packages more commonly use mailing lists and dedicated forums.

The overall surface usability impression scores range from 5 to 10, with a mean of 7.3. Gazebo (10), Drake (9), MoveIt! (9), and ROS 2 (9) score highest; they pair multiple support channels with thorough user, developer, and API documentation. Newton Dynamics (5) and GraspIt! (5) score lowest.

5.7 Maintainability

Maintainability assesses the effort required to modify a software system or component. Table 5.5 summarizes key maintainability indicators.

Table 5.5: Maintainability indicators across 28 packages.

Indicator	**Count**	**%**	**Note**
Version number documented	28	100%	–
Code review / contribution guide	26	93%	–
Non-code artifacts present	28	100%	All have config, docs, etc
Issue tracking via GitHub or equivalent	28	100%	–
Version control via git (GitHub or BitBucket)	28	100%	27 on GitHub, ODE on BitBucke

All 28 packages use git for version control. Twenty-seven of the 28 host their primary repository on GitHub; ODE is the exception, hosting on BitBucket alongside a GitHub mirror. All 28 use GitHub Issues or the equivalent (BitBucket Issues for ODE) for issue tracking. This uniformity is partly a selection effect, since the Domain X methodology expresses a preference for GitHub-style repositories because they facilitate consistent measurement; it also reflects the near-universal adoption of git and GitHub in the contemporary robotics software community.

The percentage of identified issues that are closed ranges from 34.6% (Flightmare) to 100% (DART, Newton Dynamics), with a mean of 79.1% and a median of 82.0% across the 27 packages that yielded a measurable closure rate (ODE returns no value because the BitBucket API call did not populate this field, as noted under Table 5.8). A high closure rate can indicate active issue management, but it can also reflect a low rate of new issue reporting or a practice of closing stale issues aggressively. These percentages should be interpreted with caution.

The percentage of code that consists of comments ranges from 1.1% (Project Chrono) to 28.0% (OROCOS), with a mean of 13.2% across the 27 packages with comparable single-repository measurements. The ROS 2 entry is excluded from this range and mean because its multi-repository structure does not yield a comparable single-repository code-line count (Table 5.8 marks it as not measured rather than as zero). OROCOS (28.0%), OMPL (27.4%), OpenRTM-aist (26.3%), MoveIt! (26.1%), and Simbody (25.3%) lead in comment density. Project Chrono (1.1%), Webots (1.9%), GraspIt! (2.1%), MORSE (2.7%), and Bullet/PyBullet (2.8%) have the lowest comment percentages. The Domain X methodology treats 10% as a threshold for the maintainability impression calculator.

The overall maintainability impression scores range from 4 to 10, with a mean of 7.5. Gazebo (10), Drake (9), MoveIt! (9), Pinocchio (9), and ROS 2 (9) score highest. GraspIt! (4) and Flightmare (4) score lowest, weighed down by their dead or very low activity status and limited contribution infrastructure.

5.8 Reusability

Reusability assesses the extent to which software components can be used in problem solutions other than the one for which they were originally developed. The Domain X template measures reusability through two indicators: the number of code files (as a rough proxy for component granularity) and the presence of API documentation.

The number of code files varies widely across the 27 packages with a comparable single-repository count, from 118 (Flightmare) to 7,823 (Newton Dynamics), with a median of 1,266 (Pinocchio at the midpoint of the sorted list). ROS 2 is again excluded from this distribution because its multi-repository structure does not yield a single-repository file count comparable to the rest. All 28 packages provide API documentation; where the measurement records identify documentation-generation tools, the most common are Doxygen and Sphinx.

The overall reusability impression scores range from 5 to 10, with a mean of 7.6. Gazebo (10), MuJoCo (9), OMPL (9), Pinocchio (9), ROS 2 (9), and MoveIt! (9) score highest. These packages are designed for integration into larger robotics systems or serve as the foundation that other tools build upon. MORSE (5) and GraspIt! (5) score lowest.

5.9 Surface Understandability

Surface understandability is assessed through a review of 10 randomly selected source files per package, examining code formatting, identifier quality, comment practices, and modularization. This is a shallow check, not a comprehensive code review.

Table 5.6 summarizes the surface understandability indicators.

Table 5.6: Surface understandability indicators across 28 packages.

Indicator	**Packages**	**%**
Consistent indentation and formatting style	28	100%
Explicit identification of a coding standard	14	50%
Consistent, distinctive, and meaningful identifiers	28	100%
Hard-coded constants (other than 0 and 1) present	28	100%
Comments are clear and describe what is being done	28	100%
Name/URL of algorithms used is mentioned	28	100%
Code is modularized	28	100%

All 28 packages exhibit consistent indentation and formatting, meaningful identifiers, clear comments, and modularized code organization. These are baseline indicators of professional software development practice, and their universal presence is a positive signal for the domain.

Fourteen of 28 packages (50%) explicitly identify a coding standard. This is stronger than the MI comparison point, where code style guides were rare Smith et al. [2024a], but it still leaves half of the measured MPS packages without an

explicit coding-standard artifact. Packages that do identify a coding standard include Gazebo, Webots, Bullet/PyBullet, MuJoCo, ROS 2, Drake, OMPL, Pinocchio, MoveIt!, Habitat, Project Chrono, MRPT, YARP, and OpenRTM-aist.

The overall surface understandability impression scores range from 5 to 10, with a mean of 7.3. Gazebo (10), OMPL (9), and MoveIt! (9) score highest. GraspIt! (5) scores lowest.

5.10 Visibility and Transparency

Visibility and transparency assess the extent to which a project's development process and status are visible to external observers. Table 5.7 summarizes the key indicators.

Table 5.7: Visibility and transparency indicators.

Indicator	**Packages**	**%**
Development process defined (CI, contributing guide, etc.)	25	89%
Documents recording development process and status	28	100%
Development environment documented	28	100%
Release notes published	27	96%

Twenty-five of 28 packages have a defined development process, typically manifested through CI/CD workflows, contributing guidelines, pull request templates, and issue templates. All 28 packages use GitHub Issues or similar tools to record development status. All 28 document their development environment (build dependencies, toolchain requirements, etc.). Twenty-seven of 28 publish release notes.

Three packages are recorded as unclear for the defined-development-process metric: CoppeliaSim, ODE, and OMPL. For CoppeliaSim, the mixed licensing model means parts of the development process are not publicly visible. For ODE, the unclear result is consistent with its low activity level and sparse process documentation. For OMPL, the result should be read narrowly as the outcome of this measurement item, not as a claim that the project lacks public development infrastructure in every respect.

The overall visibility and transparency impression scores range from 5 to 10, with a mean of 7.4. Gazebo (10), Drake (9), ROS 2 (9), OMPL (9), and MoveIt! (9) score highest. These packages publish thorough documentation of their development workflows. MORSE (5), Flightmare (5), Newton Dynamics (5), and GraspIt! (5) score lowest, consistent with their dead or low-activity status.

5.11 Repository Metrics

Table 5.8 presents key repository metrics for all 28 packages, including GitHub popularity indicators and codebase size measures. All data is from May 2026.

Table 5.8: Repository metrics for the 28 selected packages (May 2026).

Software	Stars	Forks	Commits	Code Lines	% Comments	% Issues Closed
AirSim	18,159	4,889	3,562	134,331	10.3%	80.0%
Bullet/PyBullet	14,465	3,074	9,405	1,528,236	2.8%	87.3%
CARLA	13,941	4,559	6,713	138,801	9.4%	82.0%
ROS 2	5,475	903	309	—	—	86.2%
MuJoCo	13,440	1,501	4,676	349,490	6.7%	91.7%
Drake	4,025	1,366	34,786	705,686	20.5%	90.0%
Webots	4,345	2,025	15,746	581,442	1.9%	87.9%
Gazebo	1,337	411	7,500	193,808	11.9%	56.7%
PhysX (OSS)	4,532	614	40†	1,030,524	17.3%	67.9%
Pinocchio	3,354	537	13,081	185,472	11.9%	92.3%
Habitat	3,662	530	1,664	143,150	7.7%	75.7%
Proj. Chrono	2,836	595	19,510	657,303	1.1%	96.8%
Simbody	2,521	492	5,041	259,447	25.3%	59.5%
MRPT	2,133	658	10,231	504,916	9.4%	95.4%
OMPL	2,047	690	5,341	105,532	27.4%	93.2%
MoveIt!	1,803	744	9,462	159,789	26.1%	80.5%
Flightmare	1,353	400	139	14,543	20.1%	34.6%
SOFA	1,194	352	27,404	349,961	4.5%	64.1%
DART	1,081	300	6,727	339,389	9.2%	100.0%
CoppeliaSim	146	46	1,206	292,460	5.1%	95.5%
OROCOS	879	443	1,229	18,063	28.0%	78.0%
OpenRAVE	801	355	10,375	623,306	19.2%	42.9%
YARP	590	214	19,020	843,982	15.1%	82.3%
ODE	—	—	2,119	147,055	18.8%	—
GraspIt!	213	86	7377	110,614	2.1%	56.6%
OpenRTM-aist	24	14	4,360	86,965	26.3%	81.4%

Notes: ROS 2 code line and comment counts are not directly comparable because the ROS 2 organization spans hundreds of repositories. ODE is hosted on BitBucket; star, fork and issue-closure metrics are reported as "—" because the BitBucket APIs used during measurement did not return values for these fields under our access configuration, not because they are conceptually unavailable on that platform. The "code lines" column reports SCC-measured code lines in the primary measured repository. † For PhysX, the commit count reflects only the public open-source SDK release on `NVIDIA-Omniverse/PhysX`; the broader internal NVIDIA codebase is not publicly accessible and is therefore not represented in this metric.

GitHub stars (a rough proxy for community attention) span almost three orders of magnitude, from 24 (OpenRTM-aist, Newton Dynamics) to 18,159 (AirSim). The median is approximately 2,300. The correlation between stars and measurement scores is not straightforward. AirSim leads in stars by a wide margin but does not appear among the top-scoring packages in most quality categories. Conversely, Gazebo scores highly across most quality categories but has relatively modest star counts (1,337), likely because its community engagement predates the current GitHub repository organization and because its user base interacts with it primarily through ROS rather than directly through its GitHub page.

The number of commits ranges from 139 (Flightmare, which is now inactive) to 34,786 (Drake). Commit activity in the last 12 months shows a similar range: Drake leads with sustained high monthly activity, while several dead packages (MORSE, GraspIt!, Flightmare) show zero or near-zero recent commits.

The percentage of code that is comments shows substantial variation, from 1.1% (Project Chrono) to 28.0% (OROCOS). The Domain X maintainability calculator

uses 10% as a threshold; 15 of the 27 packages with comparable measurements (about 56%) exceed it. The packages below 10% include some of the largest codebases in the study (Bullet/PyBullet at 2.8%, SOFA at 4.5%, Project Chrono at 1.1%), so codebase size alone does not determine comment density.

5.12 Overall Observations

Table 5.9 summarizes the overall impression scores across all nine quality categories for each package, along with a simple unweighted per-package mean. The scores reported here are the raw 1–10 impression values assigned during measurement using the Domain X Appendix C calculator. The unweighted mean treats all nine qualities as equally important; a weighted ranking using the Analytic Hierarchy Process, as described in Chapter 3, would account for differential importance across qualities but is not computed in this report (Chapter 2, Section 2.6).

Table 5.9: Overall impression scores by quality category (1–10 scale) with an unweighted per-package mean. I = Installability, C = Correctness, SR = Surface Reliability, SRo = Surface Robustness, SU = Surface Usability, M = Maintainability, Re = Reusability, SUnd = Surface Understandability, VT = Visibility/Transparency. The Mean column is the simple arithmetic mean of the nine category scores. The Domain X methodology supports a weighted ranking via the Analytic Hierarchy Process (Chapter 2, Section 2.6); that weighted ranking is not computed in this report.

Software	**I**	**C**	**SR**	**SRo**	**SU**	**M**	**Re**	**SUnd**	**VT**	**Mean**
AirSim	8	8	8	8	8	8	8	7	7	7.8
Bullet/PyBullet	9	7	8	7	7	7	8	7	7	7.4
CARLA	7	8	8	7	8	8	8	7	8	7.7
CoppeliaSim	8	8	8	7	7	7	8	7	7	7.4
DART	8	8	8	7	7	8	8	7	7	7.6
Drake	7	10	10	9	9	9	8	8	9	8.8
Flightmare	7	6	7	8	6	4	6	6	5	6.1
Gazebo	10	10	10	10	10	10	10	10	10	10.0
GraspIt!	5	5	6	5	5	4	5	5	5	5.0
Habitat	8	8	8	8	8	8	8	8	8	8.0
MORSE	9	8	8	8	8	8	5	6	5	7.2
MoveIt!	7	9	9	8	9	9	9	9	9	8.7
MRPT	8	8	7	8	7	8	8	8	8	7.8
MuJoCo	7	7	8	8	8	8	9	8	8	7.9
Newton Dyn.	6	5	6	6	5	5	6	6	5	5.6
ODE	4	6	7	7	6	6	7	7	7	6.3
OMPL	9	9	9	8	8	8	9	9	9	8.7
OpenRAVE	8	8	8	7	8	8	7	8	7	7.7
OpenRTM-aist	7	8	7	7	6	7	6	7	7	6.9
OROCOS	7	8	7	6	6	6	7	6	6	6.6
PhysX (OSS)	4	6	6	6	7	8	8	8	8	6.8
Pinocchio	10	9	9	8	8	9	9	8	8	8.7

A handful of packages anchor the top of the table. Gazebo scores 10 in all nine categories and is the most consistently high-rated package in the study. Drake and MoveIt! score 9 or 10 across most categories, and ROS 2, OMPL, and Pinocchio sit just below them. These packages share several traits: established stewardship (large organizations or active research groups), continuous contributor activity, comprehensive documentation, automated testing wired into CI/CD, and published contribution guidelines and release notes. None of these traits is individually decisive, but the combination separates the top tier from the rest.

The bottom of the table tracks maintenance status, but only loosely. GraspIt! (last commit July 2020), Flightmare (last commit May 2023), and MORSE (last commit March 2022) appear in the lower tier for most quality categories. Newton Dynamics, though technically alive at the May 2026 measurement date, joins them, reflecting its largely individual-maintained codebase and thin documentation infrastructure. OpenRAVE is also classified as dead under the 18-month rule (last commit August 2024), but its quality scores cluster in the middle of the table rather than the bottom; the documentation, test infrastructure, and codebase organization built during years of active development have not visibly degraded in the period since. In short, packages that go inactive tend to accumulate outdated documentation, broken installation procedures, and unresolved issues, but the rate at which they do so depends on the engineering capital accumulated before they went quiet.

Installability is generally strong across the domain. All 28 packages provide installation instructions and some form of automation, and only 2 broke during installation. This compares favourably with prior State-of-the-Practice studies, where the Medical Imaging study reported successful installation for only about half of the measured

packages Smith et al. [2024a]. The widespread adoption of standard package managers (APT, pip, Conda), containerization (Docker), and CMake-based builds in the robotics community is the most plausible explanation.

Documentation is universally present but varies in depth. All packages provide tutorials, user manuals, and API documentation. Only one package documents expected user characteristics, and only 14 of 28 explicitly identify a coding standard. The same gaps appear in the medical imaging and LBM samples Smith et al. [2024a,b], so they look more like patterns of research software practice than peculiarities of MPS.

Repository activity is concentrated in a small subset of packages: Drake, SOFA, Project Chrono, YARP, and Pinocchio account for a disproportionate share of total commits, reflecting sustained multi-year development by well-resourced teams. At the other extreme, several packages show minimal activity in the last 12 months.

The relationship between community popularity and measurement scores is not linear. Section 5.13 reports the explicit ranking comparison. Star counts accumulate over years and capture cumulative attention; the quality scores capture what each project's documentation, tests, and release infrastructure look like today, so the two indicators answer different questions and need not agree.

These observations are descriptive and depend on the specific measurement procedure, time frame, and interpretation rules described in Chapter 3. They are not claims that any single package is universally superior. Chapter 6 interprets these results against prior State-of-the-Practice studies, and Chapter 9 answers the research questions in light of the evidence.

5.13 Comparison with Community Indicators

The Domain X quality mean reported in Section 5.12 reflects the per-package state of documentation, testing, release management, and related software engineering practices in May 2026. Community indicators such as GitHub stars, forks, and recent commit activity capture a different signal: cumulative visibility, user adoption, and word-of-mouth attention built up over time. The two need not agree. This section presents both rankings side by side, using the per-package quality mean from Table 5.9 and the star count from Table 5.8, and identifies where the indicators converge and where they diverge.

Table 5.10: Quality-mean rank versus GitHub stars rank for the 28 selected packages. $\Delta = S - Q$: positive values indicate a higher quality rank than stars rank (under-recognized for its quality); negative values indicate a higher stars rank than quality rank (popular without proportional quality scores). Ties receive the same rank. ODE is hosted on BitBucket and is excluded from the stars rank, consistent with Table 5.8.

Software	**Q-Mean**	**Q-Rank**	**Stars**	**S-Rank**	**Δ**
Gazebo	10.0	1	1,337	17	+16
Drake	8.8	2	4,025	8	+6
MoveIt!	8.7	3	1,803	15	+12
OMPL	8.7	3	2,047	14	+11
Pinocchio	8.7	3	3,354	10	+7
ROS 2	8.7	3	5,475	5	+2
Habitat	8.0	7	3,662	9	+2
MuJoCo	7.9	8	13,440	4	−4
AirSim	7.8	9	18,159	1	−8
MRPT	7.8	9	2,133	13	+4
Webots	7.8	9	4,345	7	−2
CARLA	7.7	12	13,941	3	−9
OpenRAVE	7.7	12	801	21	+9
DART	7.6	14	1,081	19	+5
Bullet/PyBullet	7.4	15	14,465	2	−13
CoppeliaSim	7.4	15	146	25	+10
Proj. Chrono	7.4	15	2,836	11	−4
YARP	7.3	18	590	22	+4
MORSE	7.2	19	367	23	+4
Simbody	7.0	20	2,521	12	−8
OpenRTM-aist	6.9	21	24	26	+5
PhysX (OSS)	6.8	22	4,532	6	−16
SOFA	6.8	22	1,194	18	−4

The 28 packages fall into four broad patterns. **Rough agreement at the top.** Drake ($\Delta = +6$), ROS 2 ($\Delta = +2$), Pinocchio ($\Delta = +7$), and Habitat ($\Delta = +2$) appear in the top tier on both indicators; well-resourced projects with sustained development tend to combine strong engineering practice with substantial visibility. **High quality, low visibility.** Gazebo ($\Delta = +16$), OMPL ($\Delta = +11$), and MoveIt! ($\Delta = +12$) score at or near the top on quality but sit well below their quality rank on stars. The Gazebo gap is partially explained at the end of Section 5.11: Gazebo's user base interacts with it primarily through ROS rather than its GitHub repository directly, so star counts undercount its reach. OMPL and MoveIt! are libraries embedded inside larger workflows and may attract attention as dependencies rather than as standalone projects. **High visibility, mid-pack quality.** AirSim ($\Delta = -8$), Bullet/PyBullet ($\Delta = -13$), CARLA ($\Delta = -9$), and PhysX (OSS) ($\Delta = -16$) sit at or near the top on stars but in the middle or lower tier on the quality mean. These projects share characteristics: broad user bases drawn from adjacent fields (gaming, autonomous-driving research, real-time physics), corporate or game-engine origins, and uneven research-software quality practice. **Consistently low.** GraspIt! ($\Delta = -4$) and Newton Dynamics ($\Delta = -1$) cluster near the bottom on both indicators; inactive or narrowly used projects tend to accumulate gaps in documentation, releases, and visibility together. OpenRTM-aist sits near the bottom on both indicators as well, but with $\Delta = +5$ its quality rank (21) is modestly better than its stars rank (26), so it belongs in this group only in the sense of low absolute standing on both axes, not as a case where popularity outruns quality.

These patterns are consistent with the finding from the Medical Imaging State-of-the-Practice study that GitHub popularity and measurement-based rankings do

not align in a simple way Smith et al. [2024a]. They support the broader argument that community-facing indicators of visibility and adoption are not interchangeable with measured software engineering quality; treating stars as a quality proxy obscures meaningful differences across packages.

Chapter 6

Comparison with Other Research Software Domains

This chapter compares the findings for motion planning and simulation (MPS) software with prior State-of-the-Practice studies in other research software domains, especially Medical Imaging (MI) software Smith et al. [2024a] and Lattice Boltzmann Method (LBM) software Smith et al. [2024b]. The goal is not to claim that one domain is better than another. The domains differ in age, user communities, computational goals, and measurement dates. Instead, the comparison identifies where MPS software follows patterns already observed in research software and where it appears to differ.

The comparison follows the structure used in the LBM study. That study compared LBM projects with other research software in terms of repository artifacts, tool usage, and development principles, processes, and methodologies Smith et al. [2024b]. The same three dimensions map directly to RQ3, RQ4, and RQ5 in this report.

6.1 Comparison of Repository Artifacts

RQ3 asks how MPS projects compare to other research software domains with respect to the artifacts present in their repositories. In the LBM study, repository artifacts were grouped by frequency and compared with research software development guidelines Smith et al. [2024b]. The same idea applies here: artifacts such as installation instructions, user manuals, API documentation, issue trackers, tests, release notes, and contribution guides make development activity visible and make software easier to install, inspect, reuse, and maintain.

Table 6.1 summarizes the main artifact observations from Chapter 5. These counts are based on the May 2026 measurement snapshot.

Table 6.1: Summary of selected artifacts and visible development evidence in MPS packages.

Artifact or evidence type	**Packages**	**%**
Public source repository	28/28	100%
Installation instructions	28/28	100%
Installation automation	28/28	100%
Getting-started tutorial	28/28	100%
User manual	28/28	100%
API documentation	28/28	100%
Issue tracking	28/28	100%
Version control	28/28	100%
Unit tests visible or available	27/28	96%
Release notes	27/28	96%
Contribution or code-review guidance	26/28	93%
Explicit coding standard	14/28	50%
Documented expected user characteristics	1/28	4%

The main artifact finding for MPS software is the near-universal presence of user-facing and developer-facing infrastructure. Every measured package provides installation instructions, some form of installation automation, a getting-started tutorial, a user manual, API documentation, issue tracking, and version control evidence. MPS compares well with prior State-of-the-Practice studies on this dimension. The MI study reported that 22 of 29 packages provided a user manual Smith et al. [2024a], while all 28 MPS packages do so in the current measurement. The LBM study found

that installation guides and tutorials were common artifacts but API documentation was rare in the measured LBM sample Smith et al. [2024b]; in this study, every MPS package provides API documentation.

This difference likely reflects the way MPS software is used. Many MPS projects are not stand-alone research scripts; they are simulators, middleware systems, planning libraries, physics engines, or integration frameworks. Their users often need to install them into larger toolchains, call their APIs from other software, and combine them with robotics ecosystems such as ROS. That use pattern creates pressure for installation documentation, tutorials, and API references. The presence of these artifacts does not by itself prove high quality, but it does show that the MPS ecosystem has adopted many of the artifacts that make research software inspectable and reusable.

The comparison is less uniformly positive for formal software engineering artifacts. Only one MPS package documents expected user characteristics, and 14 of 28 explicitly identify a coding standard. The coding-standard result is stronger than in the MI and LBM comparison studies, but it still means that half of the measured MPS packages do not expose this artifact. The LBM study observed that research software projects often fall short of recommended artifacts such as roadmaps, coding standards, checklists, uninstall instructions, and formal design or requirements documentation Smith et al. [2024b]. The MPS results follow the same broad pattern: practical artifacts that help users install, run, and call the software are widespread, while artifacts that record design intent, expected user assumptions, and explicit development rules are less common.

The MPS artifact profile therefore appears stronger than MI and LBM in several

public-facing documentation categories, especially manuals and API documentation. Still, it does not eliminate the usual gap between research software practice and software engineering recommendations. MPS packages are generally well equipped for use and integration, but their repositories less often expose the rationale, process rules, and user assumptions behind that software.

6.2 Comparison of Tool Usage

RQ4 asks how MPS projects compare to other research software domains with respect to tool usage. In the LBM study, tool usage was discussed in terms of development tools, dependencies, and project management tools, with special attention to version control and continuous integration Smith et al. [2024b]. The current measurement provides strong evidence for version control, issue tracking, build or installation automation, testing, API documentation, and visible development infrastructure. Evidence for documentation-generation tools is narrower, since these tools are recorded for 12 of 28 packages.

Version control and issue tracking provide the most direct tool comparison. All 28 MPS packages use git-based version control and host public repositories on GitHub or an equivalent public platform. All 28 packages also use GitHub Issues or an equivalent issue tracker. The LBM study reported version control for 67% of all LBM packages and 73% of alive LBM packages Smith et al. [2024b]. The MPS result is higher, although part of that difference comes from the current study design: repository-based measurement requires public repository evidence, so packages without suitable public repositories are filtered out before measurement. Even with that caveat, the result shows that the measured MPS ecosystem is strongly aligned with contemporary

repository-centered development.

Build and installation automation are also widespread in the MPS sample. All 28 packages provide some form of installation automation, such as build files, scripts, package-manager support, or container-based installation support. This matters because MPS packages often depend on compiled C++ code, Python bindings, physics libraries, graphics systems, middleware, and operating-system packages. Manual installation would make such systems difficult to reproduce. The strong installability result in Chapter 5 suggests that MPS maintainers have invested in build and installation tooling to reduce this burden.

Testing and verification tooling are also visible. Unit tests are available for 27 of 28 packages, automated testing is recorded for 26 packages, assertions are recorded for 23 packages, and documentation-generation tools are recorded for 12 packages. This matters for the LBM comparison because the LBM study treated test cases as an uncommon artifact and discussed automated testing as one of the prerequisites for effective continuous integration Smith et al. [2024b]. In the MPS sample, testing evidence is much more common. This does not mean that every package has complete test coverage, nor that the tests exercise realistic robotics workloads. It does show, however, that test infrastructure is a visible part of most measured MPS projects.

Continuous integration should be interpreted more cautiously. Chapter 5 records whether packages have a defined development process, often visible through CI/CD workflows, contribution guides, pull request templates, or issue templates. Twenty-five of 28 packages have such visible development-process evidence. This does not mean that 25 packages all use continuous integration in the same way or to the same depth. Some may use CI for builds, others for tests, documentation, static checks,

packaging, or release automation. The narrower conclusion is that MPS repositories often expose automation and workflow infrastructure, but this report does not treat CI maturity as a separately ranked measure.

API documentation is another point of difference. The MPS measurement found API documentation for all 28 packages. This should be separated from tool usage: documentation-generation tools such as Doxygen or Sphinx are explicitly recorded for 12 MPS packages, not for the full set. The LBM paper reported that roughly half of the LBM packages mentioned documentation-generation tools and that API documentation was rare Smith et al. [2024b]. The MPS result is stronger on API documentation itself, likely because many MPS packages are libraries, frameworks, or middleware systems whose adoption depends on programmable interfaces.

Overall, MPS projects appear to use modern repository and development tools consistently in the categories visible from public repositories. The best-supported observations concern version control, issue tracking, installation automation, tests, and API documentation. Compared with MI and LBM, the clearest advantages are API documentation and testing evidence; basic repository infrastructure is at least comparable with MI and stronger than the LBM comparison point. The weaker point is not the absence of tools, but the uneven visibility of how tools are governed: coding standards, process rules, review expectations, and user assumptions are not documented as consistently as the tools themselves.

6.3 Comparison of Principles, Processes, and Methodologies

RQ5 asks how MPS projects compare to other research software domains with respect to principles, processes, and methodologies. This question is harder to answer from public artifacts alone because process is partly social. Repositories can show issue trackers, pull requests, release notes, contribution guides, and automated workflows, but they cannot fully reveal how teams make design decisions, how maintainers negotiate priorities, or how much informal review occurs outside the public repository.

Within that limitation, MPS projects show more visible process structure than many research software projects described in prior studies. Twenty-five of 28 MPS packages have evidence of a defined development process, typically through CI/CD workflows, contribution guides, pull request templates, issue templates, or similar repository artifacts. Twenty-seven of 28 publish release notes. Twenty-six of 28 provide contribution or code-review guidance. These artifacts indicate that many measured MPS projects expect external users or contributors and have created at least minimal structures for managing that interaction.

This contrasts with the LBM discussion of principles and process, where process evidence was often informal and was strengthened by interviews with developers Smith et al. [2024b]. LBM developers described small-team practices, lead developer roles, informal peer review, and issue tracking. The current MPS study does not yet have approved interview findings, so it cannot make equivalent claims about internal team behaviour. What it can say is that public MPS repositories frequently expose the artifacts associated with open collaboration: issues, pull requests,

contribution instructions, release records, and workflow files.

The MPS process picture is still incomplete in two ways. First, 14 of 28 packages explicitly identify a coding standard, so this artifact is more common in MPS than in the comparison studies but still absent from half of the sample. Second, only one package documents expected user characteristics. This limits traceability between the intended audience and the design of tutorials, APIs, examples, and support channels. These gaps echo the LBM paper's observation that research software often has practical documentation but weaker formal requirements and design rationale Smith et al. [2024b].

The heterogeneity of MPS software also affects process comparison. A full simulator such as Gazebo or Webots, a middleware platform such as ROS 2, and a planning library such as OMPL do not have identical development needs. Large ecosystem projects tend to expose more formal governance, release management, and contribution infrastructure. Smaller or older packages may provide source code and documentation but little explicit process description. For that reason, the comparison should not treat the 28 MPS packages as interchangeable products. The commonality analysis in Chapter 4 is important because it explains why different process expectations may be reasonable for different kinds of MPS software.

Taken together, the evidence suggests that MPS projects are relatively mature in visible, repository-based process infrastructure. They align with modern open-source practice through public issue tracking, version control, release notes, contribution guidance, and automation. They still resemble other research software domains in the weaker documentation of design rationale, user assumptions, coding standards, and formal process principles.

6.4 Summary of Cross-Domain Findings

The cross-domain comparison answers RQ3–RQ5 as follows.

For RQ3, MPS projects compare favourably with prior research software domains in the presence of repository artifacts. Installation instructions, installation automation, tutorials, user manuals, API documentation, issue tracking, version control, unit tests, release notes, and contribution guidance are common or near-universal in the measured MPS set. Compared with MI and LBM, the MPS sample appears especially strong in user manuals, API documentation, and testing evidence. Its basic repository infrastructure is comparable with MI and stronger than the LBM comparison point. The main gaps are formal artifacts: documented user characteristics, requirements-style information, design rationale, and explicit coding standards for the half of packages that do not record one.

For RQ4, MPS projects show strong tool adoption in the categories visible from public repositories. Version control and issue tracking are universal in the measured set, installation automation is universal, unit-test evidence is present in nearly all packages, and API documentation is present for every package. Read together, these numbers indicate that MPS software development is tightly connected to modern open-source tooling. The comparison should be read with care because the study filters for public, repository-measurable software; proprietary or closed-core robotics tools may follow different practices.

For RQ5, MPS projects show substantial public evidence of development processes, but the evidence is mostly artifact-based. Contribution guidance, release notes, issue tracking, and visible workflow infrastructure are common. More formal principles and process documentation are less consistent. This pattern places

MPS software between two tendencies: it is more visibly tooled and documented than some prior research software domains, but it still shares the broader research software tendency to under-document design intent, requirements assumptions, and coding rules.

Taken as a whole, the measured MPS ecosystem shows a solid public-artifact profile. Its strengths are practical and integration-oriented: installability, tutorials, API documentation, version control, issue tracking, testing evidence, and release visibility. Its weaknesses are also familiar from prior research software studies: formal requirements, explicit user assumptions, coding standards, and design rationale remain less consistently documented. That mixed picture is the substance of this report's contribution: it characterizes the state of the practice in MPS software rather than recommending a single package as superior for all robotics tasks.

Chapter 7

Developer Interviews: Planned Component

Developer interviews are part of the Domain X State-of-the-Practice methodology Smith et al. [2021]. They provide qualitative evidence that repositories cannot: the reasons behind development choices, what obstacles developers actually run into, and how teams handle problems that leave no trace in public artifacts.

No approved interview data is available for this study. This chapter describes the planned interview component as a record of methodology and a guide for future work. RQ6–RQ9 remain open questions pending ethics approval and data collection.

7.1 Purpose of the Interviews

Public repositories show what a project has produced, but not why particular choices were made. A team might rely on informal code review, use internal CI tooling invisible to outsiders, or follow conventions that are never written down. Interviews

surface these practices and, more usefully, identify the obstacles developers encounter that would not appear anywhere in the public record.

The interview component addresses RQ6 through RQ9: developer pain points in MPS software, how those pain points compare across research software domains, what best practices address them, and what the broader research software community might learn from MPS developers. These findings would complement the artifact-based evidence in Chapters 4 and 5.

7.2 Planned Protocol

The Domain X interview guide uses 20 semi-structured questions covering the interviewee's background, project organization and user base, development and documentation practices, and recurring difficulties. The semi-structured format allows follow-up on topics that participants raise.

For the MPS domain, the protocol must account for the different roles of tools in the study set. A question about usability or documentation means something different for a full simulator than for a lower-level planning library or middleware framework.

Recruitment and reporting must follow the applicable McMaster ethics approval and consent requirements. No participant responses, identifying details, or interview findings may be included unless the relevant ethics materials, consent records, and approved summaries are available.

7.3 Analysis Approach

Interview responses would be analyzed qualitatively, coding for recurring pain points, reported development practices, and strategies developers use to address quality concerns. Where possible, the interview evidence would be connected to the repository and measurement data, with explicit distinction between what is publicly observable and what participants report.

Chapter 8

Threats to Validity

Any study based on publicly available artifacts is constrained by what those artifacts can reveal. The threats below concern how the candidate set was assembled, how the measurements were made, and what the resulting scores can reasonably claim.

8.1 Candidate Discovery

The candidate set was built from academic survey papers, community-curated lists, and LLM-assisted discovery. Source appearance counts show how often a package turns up across those sources, not how good the software is. A package with high mention counts may simply be older or more broadly targeted; a technically strong tool with a narrower audience could score low. Any package absent from all three source types would not enter the candidate pool, whatever its actual quality.

8.2 Measurement Consistency

All measurements are based on public evidence collected in May 2026 and reflect the state of each repository at that moment. The same project measured a year earlier or later might score differently. Some quality questions also require judgement—whether documentation is adequate, for example—and two evaluators reading the same artifacts could reach different conclusions. Where judgement calls were made, the rationale is recorded in the measurement template.

8.3 Snapshot-Based Repository Data

Metrics like star counts, fork counts, open issues, commit frequency, and lines of code were recorded in May 2026 and reflect that point in time only. They change continuously. A project that appears inactive at the measurement date may have had a burst of development beforehand, or may be in a stable maintenance phase rather than in decline.

8.4 Use of LLM-Assisted Candidate Sources

LLM-generated candidate lists supplemented the academic and community-curated sources but did not drive them. These lists reflect the training data of the model rather than an independent survey of the domain, so a package unknown to the model could be absent from its output. The mention-count ranking aggregated appearances across all three source types, so a package surfacing in only the LLM-generated lists would receive a lower mention count and would be filtered out at the open-source-availability

and activity stages before measurement; the procedure does not, however, guarantee that every retained package was independently corroborated by an academic or community-curated source.

8.5 Scope Boundaries

Deciding which tools fall inside or outside the scope of MPS software involves judgement. The criteria are documented in Chapter 3, but any boundary choice shapes the candidate pool and therefore the scope of valid comparisons. Tools excluded from the study—proprietary platforms, discontinued projects, narrowly scoped libraries—may differ systematically from those that were measured.

Chapter 9

Conclusions

9.1 Summary

9.2 Answers to Research Questions

9.3 Key Findings

9.4 Future Work

Appendix A

Full Candidate Software List

Appendix B

Mention Count Source List

Appendix C

Measurement Template

Appendix D

Excluded Software Packages

Appendix E

Interview Materials

Bibliography

Howie Choset, Kevin M. Lynch, Seth Hutchinson, George Kantor, Wolfram Burgard, Lydia E. Kavraki, and Sebastian Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, Cambridge, MA, 2005.

Jack Collins, Shelvin Chand, Anthony Vanderkop, and David Howard. A review of physics simulators for robotic applications. *IEEE Access*, 9:51416–51431, 2021. doi: 10.1109/ACCESS.2021.3068769.

Jo Erskine Hannay, Carolyn MacLeod, Janice Singer, Hans Petter Langtangen, Dietmar Pfahl, and Greg Wilson. How do scientists develop and use scientific software? In *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pages 1–8, 2009. doi: 10.1109/SECSE.2009.5069155.

Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, New York, NY, USA, 2006.

Peter Michalski. State of the practice for lattice boltzmann method software. M.eng. thesis, McMaster University, 2021.

David Lorge Parnas. On the design and development of program families. *IEEE Transactions on Software Engineering*, SE-2(1):1–9, 1976.

Prakash Prabhu, Thomas B. Jablin, Arun Raman, Yun Zhang, Jialu Huang, Hanjun Kim, Nick P. Johnson, Feng Liu, Soumyadeep Ghosh, Stephen Beard, Taewook Oh, Matthew Zoufaly, David Walker, and David I. August. A survey of the practice of computational science. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, pages 1–12, New York, NY, USA, 2011. ACM. doi: 10.1145/2063348.2063374.

Thomas L. Saaty. *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. McGraw-Hill Publishing Company, New York, New York, 1980.

Spencer Smith, Yue Sun, and Jacques Carette. Statistical software for psychology: Comparing development practices between cran and other communities, 2018a.

Spencer Smith, Zheng Zeng, and Jacques Carette. Seismology software: State of the practice. *Journal of Seismology*, 22, 2018b. doi: 10.1007/s10950-018-9731-3.

W. Spencer Smith, D. Adam Lazzarato, and Jacques Carette. State of the practice for mesh generation and mesh processing software. *Advances in Engineering Software*, 100:53–71, 2016.

W. Spencer Smith, Adam Lazzarato, and Jacques Carette. State of the practice for gis software, 2018c.

W. Spencer Smith, Jacques Carette, Olu Owojaiye, Peter Michalski, and Ao Dong. Methodology for assessing the state of the practice for domain x. arXiv:2110.11575, 2021.

W. Spencer Smith, Ao Dong, Jacques Carette, and Michael D. Noseworthy. State

of the practice for medical imaging software. Manuscript, McMaster University, 2024a.

W. Spencer Smith, Peter Michalski, Jacques Carette, and Zahra Keshavarz-Motamed. State of the practice for lattice boltzmann method software. *Archives of Computational Methods in Engineering*, 31:313–350, 2024b. doi: 10.1007/s11831-023-09981-2.

David M. Weiss. Defining families: The commonality analysis. *Submitted to IEEE Transactions on Software Engineering*, 1997. Available at `http://www.research.avayalabs.com/user/weiss/Publications.html`.